



Module Title: Individual PG Project
Module Code: CST4599
Module Leader: Dr A R Lasebae

A Complexity-Based Analysis of Deepfake Detection Mechanisms

Student: Chetan Bissessur
Student Number: Moo838543
Supervisor: Dr Andrew Lewis-Smith

A project submitted in fulfilment of the requirements
for the degree of MSc Cyber Security & Pen Testing

Dept. of Computer Science

2024-25

Abstract

The rapid evolution in artificial intelligence (AI), machine learning and deep learning have contributed for deepfakes to emerge as a significant threat to digital authenticity, privacy and trust. These synthetic media can be used harass, blackmail or spread misinformation or commit fraud. While many detection algorithms have achieved high accuracy, their computational demands are often neglected, limiting their usefulness in real-time or resource-constrained environments. This research addresses the need to evaluate deepfake detection mechanisms not only for their effectiveness, but also for their computational efficiency.

This study compared two types of deepfake detection models namely Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN) merged with CNNs as a hybrid model. It also provided insights on transformer-based architectures for future works. The analysis involved both theoretical and empirical approaches. Time and space complexity were assessed using metrics such as memory usage, inference time, FLOPs amongst others to evaluate the performance of each model with input size. Experiments were conducted using public datasets such as FaceForensics++ and the Celeb-DF-v2. Key performance metrics included detection accuracy, precision, recall and F1-score.

The results revealed significant trade-off between detection performance and computational cost. This study identified which models are best suited for real-time applications and constrained environments, offering insights into how computational complexity impacts the practical deployment of deepfake detection systems.

List of Acronyms

AI Artificial Intelligence

DL Deep Learning

ML Machine Learning

CNN Convolutional Neural Network

RNN Recurrent Neural Network

LSTM Long Short Term Memory

ViT Vision Transformers

DF Deepfake

FF++ FaceForensics++

GAN Generative Adversarial Network

DFT Discrete Fourier Transform

DNN Deep Neural Network

DFDC Deepfake Detection Challenge

AUC Area Under Curve

NLP Natural Language Processing

FLOPs Floating-Point Operations

Table of Contents

1	Introduction	8
1.1	Keywords	8
1.2	Background	8
1.3	Problem Definition	8
1.4	Research Questions	9
1.5	Research Methodology	10
1.6	Aims	10
1.7	Objectives	10
1.8	Importance and Contribution	10
1.9	Outline of Report	10
2	Literature Review	12
2.1	Introduction	12
2.2	Background and Challenges in Deepfake Detection	12
2.3	Key Features and Techniques in Deepfake Detection	13
2.4	Deep Learning Detection Algorithms	13
2.4.1	Convolutional Neural Networks (CNN)	14
2.4.2	Recurrent Neural Networks (RNN)	14
2.4.3	Transformer-Based Models	15
2.5	Computational Complexity and Efficiency of Deepfake Detection	15
2.6	Related Works	15
2.7	Comparative Analysis	18
2.8	Research Gap	20
3	Methodology	21
3.1	Research Design and Approach	21
3.2	Algorithm Selection	21
3.3	Dataset Selection and Preprocessing	23
3.3.1	Preprocessing for XceptionNet	23
3.3.2	Preprocessing for XceptionNet-LSTM	24
3.4	Evaluation Metrics	24
4	Experiments and Results	27

4.1	Experimental Setup	27
4.2	Deepfake Detection Performance	27
4.2.1	CNN-based Algorithms	27
4.2.2	Hybrid Algorithms	28
4.3	Computational Complexity	28
4.3.1	CNN-Based Algorithms	28
4.3.2	Hybrid Algorithms	29
5	Findings and Discussion	30
5.1	Analysis of Experimental Findings	30
5.1.1	Performance Analysis	30
5.1.2	Efficiency Analysis	34
5.1.3	Trade-Off between Performance and Efficiency for Deepfake Detection Algorithms	35
5.2	Addressing Research Questions	36
5.3	Discussions and Limitations of the Study	37
6	Conclusion and Future Works	38
6.1	Conclusion	38
6.2	Future Works	38
	References	38
.1	Appendix A (Ethics form)	41
.2	Appendix B (Codes used)	42

List of Tables

2.1	Comparative analysis of related works	19
3.1	Algorithm Selection	23
3.2	Evaluation Terms Definition (Taeb and Chi, 2022)	24
3.3	Evaluation Variables Definition	25
3.4	Computational Efficiency Evaluation Metrics	25
4.1	Performance comparison of CNN on Celeb-DF and FF++ datasets	27
4.2	Performance comparison of CNN-LSTM on Celeb-DF and FF++ datasets	28
4.3	Computational Complexities of CNN	28
4.4	Computational Complexities of CNN-LSTM	29

List of Figures

5.1	CNN Model Performance Over Epochs for Celeb-DF Dataset	31
5.2	Hybrid Model Performance Over Epochs for Celeb-DF Dataset	32
5.3	CNN Model Performance Over Epochs for FaceForensics++ Dataset	33
5.4	Hybrid Model Performance Over Epochs for FaceForensics++ Dataset	34
1	Ethics form	41
2	Ethics form	41
3	Ethics form	42
4	Ethics form	42

Chapter I

Introduction

I.1 Keywords

CNN, RNN, LSTM, Deepfake, Computational Efficiency, Computational Complexity

I.2 Background

In recent years, artificial intelligence, machine learning and deep learning have advanced at an unprecedented pace, unlocking powerful capabilities and reshaping industries and everyday life. While these technologies bring significant opportunities such as smarter automation and groundbreaking innovations, they also carry a darker side. The same tools that fuel progress can be exploited to manipulate reality itself, altering images, voices and even entire videos with alarming precision. This phenomenon, known as deepfakes, poses one of the most unsettling challenges of our digital age and it can be said that a ‘deepfake pandemic’ has emerged with the easy accessibility to AI tools such as ChatGPT and Gemini nowadays. For instance, in the United Kingdom, there has been more than 100 deepfake video ads impersonating the former Prime Minister, Rishi Sunak on Facebook in early 2024. Some of these even falsely showed him making statements he never made¹.

The spread of deepfake are often carried out by malicious users to disseminate disinformation, misinformation and for other unlawful purposes. It is important to note the difference between misinformation and disinformation. The former is false information shared without the intention to do wrongful things and it can be honest mistakes due to factors such as misunderstanding or misinterpretation. In contrast, disinformation refers to false information shared in an attempt to do malicious things to people such as influencing political opinions which can even create social unrest. It is crucial that the fake contents can be discovered so as to prevent further damage and so that these technologies can be used for noble purposes. Thus, algorithmic approaches such as convolutional or recurrent neural networks, deep learning for detecting deepfakes should be explored and the performance evaluation of the approaches or algorithms must also be performed.

I.3 Problem Definition

There has been significant evolution in the fields of artificial intelligence, machine learning and deep learning over the past few years and these can be used to alter digital data (Rana, Nobi, et al., 2022) which can

¹[The Guardian, Deepfake video adverts \(2024\)](#)

be used to spread misinformation, blackmail or harass people and many other issues of public concern. Beyond these, deepfake media also facilitates financial fraud through impersonations, inflicts reputational and psychological harm on individuals. This distortion of data is referred to as deepfakes made possible through deep learning techniques which makes it hard for the human eye to know if an image, video or audio is authentic (Almars, 2021).

There are research which have been carried in the field of tackling misinformation and disinformation using AI such as in the paper (Saeidnia et al., 2025). These issues have to be tackled as they create societal unrest and challenges to information integrity which are closely related to this study as well. The more specific research carried out which demonstrates the use of various algorithms to detect deepfakes. There are traditional detection methods which were initially used such as image and video manipulation techniques. Nowadays, most mechanisms use deep learning or neural network methods including Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs) and transformer-based algorithms (Weerawardana and Fernando, 2021).

With rapid progress in the fields of AI and machine learning, it is crucial that deepfakes are detected quickly and accurately as great damage can be caused if this is not the case. Nevertheless, after going through relevant literature, it has been noted that limited research has been done on the computational efficiency² of the various deep learning algorithms such as CNN and RNN in relation to deepfake detection, especially in terms of time and space complexity³. There are some existing works carried out where deep learning models have been evaluated for deepfake detection with the primary focus on the performance of CNNs. For instance, the work (Ritter, Lucian, Chowanda, et al., 2023) did a comparative analysis of different types of CNN models for deepfake detection. Another paper, (Taeb and Chi, 2022) did comparison on the various techniques based on deep learning for deepfake detection. It is, however, noted that limited research have been done in exploring combinations of RNNs and CNNs and transformer-based algorithms for deepfake detection and only a few studies have addressed the computational complexity of these algorithms. While it is noted that many deepfake detection algorithms achieve high accuracy in controlled settings, they are often too slow or resource-intensive to monitor content at scale or in real-time. This highlights the need to do a comparison of the deepfake detection methods not only in terms of accuracy but also regarding the efficiency and scalability of the algorithms.

1.4 Research Questions

- How do CNNs-based and RNNs-based deepfake detection models compare in terms of computational complexity and accuracy ?
- To what extent do time and space complexities affect the efficiency of each algorithm ?
- Which algorithm offers the best trade-off between computational efficiency and applicability in real-world applications, and why?

²Computational efficiency measures how effectively a system uses computational resources such as time and memory to perform a task.

³The amount of time an algorithm takes to complete as a function of input size

1.5 Research Methodology

The methodology for this project consisted of quantitative research techniques. This implies dealing with the results obtained from the complexity analysis of the different algorithms chosen through the use of public datasets such as FaceForensics++ and the Celeb-DF-v2 and existing tools.

1.6 Aims

The aim of this project was to perform an analysis of various deep learning algorithms used for deepfake detection. It was based mainly on performance, computational complexity and the time and space efficiency of these techniques. It was done so that the performance and computational efficiency of each algorithm could be determined and to show the trade-off between efficiency and applicability in real-world applications.

1.7 Objectives

- To conduct a theoretical analysis of the time and space complexity of deepfake detection algorithms based on CNNs and RNNs models using relevant metrics.
- To measure and compare key performance metrics such as accuracy, memory usage, time taken for the selected models.
- To investigate the impact of input size and model architecture on the computational efficiency and detection performance of each algorithm.
- To carry out experiments to evaluate selected deepfake detection models using public datasets such as FaceForensics++ and the Celeb-DF-v2.
- To identify the most computationally efficient models suitable for real-time or resource-constrained environments and discuss the trade-off between computational cost and detection accuracy.

1.8 Importance and Contribution

Understanding the computational demands of deepfake detection algorithms is crucial as it addresses a critical gap in the field by evaluating not only the detection performance of deep learning models but also their efficiency. This dissertation carries out a complexity-based analysis of CNN and a hybrid model where CNN and RNN are combined. The focus is on time and space complexity alongside performance metrics such as accuracy, precision, recall and F1-score. The study contributes to the field by providing insights into the trade-off between model effectiveness and computational cost thus offering guidance for selecting or designing deepfake detection algorithms which are both accurate and efficient while also contributing on the theoretical side of computer science as well.

1.9 Outline of Report

This report performed a literature review on the deep learning algorithms used for deepfake detection. These include analysis of CNNs, RNNs and transformer-based models for deepfake detection. It also explained what main features and techniques are currently being used in the field and emphasis was also laid on the

computational complexity and efficiency side of deepfake detection algorithms. The related works to this study were also scrutinised and a summary of their main findings was provided before outlining the research gap. The report then explains the methodology which was used in terms of algorithm and dataset selection and the preprocessing. The evaluation metrics based on which the performance and efficiency of the algorithms were assessed were also explained. The experiments and results section then followed where the results obtained from experiments were recorded both for the performance and computational complexity. The findings were analysed and discussed as well in the report and the research questions were also addressed. This part also outlined the setbacks encountered during the project and how they were solved. The last chapter contains the conclusion and provides directions for further research which can be carried out in the field.

Chapter 2

Literature Review

2.1 Introduction

With the advancements in AI, machine learning and deep learning and their increasing availability, it is becoming easier to manipulate media for malicious purposes. This is why it is crucial to have algorithms which are effective in detecting forged media known as deepfakes so as to counter this cybercrime. The time and space complexity of deepfake detection algorithms is important as they are often slow or too resource-intensive to monitor contents at scale or in real-time even though they achieved high accuracy in controlled environments.

This chapter lays emphasis on the details of how these algorithms work with emphasis on deep learning detection algorithms. The particular types of neural networks which will be analysed are CNN, RNN and transformer-based models. This will be followed by the main features and techniques being currently used for deepfake detection. This part also explores what is computational complexity and efficiency of algorithms so as to be able to later assess the effectiveness of the algorithms being experimented with. An extensive analysis of work related to this topic will be done to better understand the issue being discussed followed by a comparative analysis of the works which will provide a summary of the existing works for better comprehension. This will then lead to a brief summary of the findings in existing literature which will allow to identify gaps before moving to the next phase of the project where the computational complexity of deep learning algorithms will be analysed.

2.2 Background and Challenges in Deepfake Detection

Deepfake technology employs advanced machine learning techniques to create highly realistic synthetic videos. These manipulated media can imitate a person's speech and facial expressions which make them a powerful tool for malicious purposes such as spreading disinformation, conducting identity theft and even blackmail (Reddy, A. P. Kumar, and Swetha, 2024). There are four main categories in which deepfakes can be divided namely identity swapping, face reenactment, attribute manipulation and full face synthesis (Akhtar, 2023). These categories may appear in both image-based or video-based deepfakes. For instance, still manipulations are used in image-based deepfakes but video-based deepfakes could include methods such as lip-syncing and full-motion manipulation. Furthermore, there can be audio-based deepfakes which em-

ploy voice cloning or synthetic speech and also multi-modal deepfakes which combine manipulated video, text and audio. Each type of these deepfakes presents unique challenges for detection as advances in generative models continue to enhance their realism and complexity.

While numerous research have focused on improving the accuracy of deepfake detection algorithms, less attention has been given to their computational efficiency. Many novel techniques such as deep learning approaches demand considerable processing power, memory and storage due to the size and complexity of the models. This raises challenges in terms of scalability, real-time detection and deployment on resource-constrained devices. As deepfakes become increasingly sophisticated, it is crucial to investigate the time and space complexity of detection algorithms to ensure they are both effective and computationally practical. To better understand these computational issues, it is necessary to analyse the key features and techniques being used in deepfake detection.

2.3 Key Features and Techniques in Deepfake Detection

There are several methods which can be used for their detection and there may be some techniques which are most suited for each category which will be discussed in this section.

The paper (Akhtar, 2023) did a summary of the various approaches used in other papers to detect each type of deepfake. For identity swapping, the main methods utilised were CNNs, multi-attentional, transformers via bag-of-feature for generalisation, perceptual image assessment and GANs and multi-modal multi-scale transformer (M2TR). The methods used for detecting reenactment included autoencoder, CNNs together with attention mechanism, knowledge distillation, U-Net structure and multistream vision transformer network. For attribute manipulation detection, the mechanisms used were CNNs, DFT together with CNNs, DNNs, human majority voting and gradient operator convolutional network with tensor pre-processing and manipulation trace attention module. The entire face synthesis deepfakes were detected through CNNs, incremental clustering, two-stream CNNs, cross-band and spatial co-occurrence matrix together with CNNs and human-annotated saliency maps into deep learning loss function. The paper (Rana, Nobi, et al., 2022) as well studied several methods of detecting deepfakes such as classical ML-based, statistical models, blockchain-based methods and deep learning-based methods. The traditional detection methods provide useful baselines, the rapid advancements and sophistication of deepfakes has highlighted the crucial role of neural networks in achieving more accurate and robust detection.

2.4 Deep Learning Detection Algorithms

Deep neural networks are now being used for deepfake detection but before that, there were traditional methods used such as image and video manipulation techniques. They worked by attempting to detect differences between a source video and the target one based mainly on features such as face swapping, similarity of face or face patches amongst others (Weerawardana and Fernando, 2021). However, nowadays novel tools such as the Generative Adversarial Network (GAN) are being used to produce realistic fake videos that make the recipient think it is true (Saikia et al., 2022). This is why the detection algorithms are now based

on deep learning which is a subset of machine learning. More specifically, there are neural networks being used mainly such as Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN).

2.4.1 Convolutional Neural Networks (CNN)

A neural network is a machine learning technique derived from the concept of mimicking the human brain (Zou, Han, and So, 2009). It is widely applied for problem solving in pattern recognition, data analysis, control and clustering (K. Kumar and Thakur, 2012). There are different types of neural networks and in this section, the application of Convolutional Neural Network (CNN) for deepfake detection will be scrutinised. CNNs are deep-learning architectures which are commonly used in computer vision tasks to identify patterns and characteristics. One major benefit of CNN is its ability to automatically extract useful information from raw data by using convolutional layers. After gathering spatial data from inputs, the layers will be applied to feature extraction. The extracted features will consequently be used to detect the deepfake through the fully connected layers in the neural network (Ritter, Lucian, Chowanda, et al., 2023).

However, CNN has not been tested thoroughly on more sophisticated deepfake creation methods such as the face-swapping method. There are very few research which achieved better accuracy in deepfake detection through deep-learning methods using CNN against custom-built features (Stephen and Mantoro, 2022). Another issue with this technique is that there are attacks which are designed to prevent detection by CNN-based methods even though it has the ability of self-learning (Ritter, Lucian, Chowanda, et al., 2023).

2.4.2 Recurrent Neural Networks (RNN)

An artificial neuron containing one or more feedback loops makes up the RNN class of supervised machine learning models (Salehinejad et al., 2017). One kind of this neural network is the long short term memory (LSTM) which is used for learning long-term dependencies in data sequences and can thus be used to detect sequence processing issues (Güera and Delp, 2018). The benefits of RNNs include its abilities to examine temporal sequences, to capture dependencies between frames and to detect temporal inconsistencies (Anjaneyulu et al., 2024).

The work in (Güera and Delp, 2018) put forward an analysis which consisted of two phases to detect deepfakes through RNN with focus on face-swapping detection. The initial stage relates to the use of CNN for the extraction of the properties at the frame-level. Following this, the RNN which is temporally-aware will be used for capturing temporal discrepancies across frames brought after the manipulation of data performed for the face-swapping. The paper (Al-Adwan et al., 2024) also proposes a similar approach to tackle the surge of deepfake media. The LSTM networks have been used in this work to classify sequential data as they are RNNs which are known to deal with sequential data better. This is because they can capture temporal dependencies in data by staying in a hidden state over time. A model was developed in the work (Jadhav et al., 2024) whereby a RNN is trained after using CNN to extract features on a frame-by-frame basis. The trained system has the capacity to identify if data in a video has been altered. This demonstrates its effectiveness in identifying temporal irregularities thus detecting if a content is a deepfake.

2.4.3 Transformer-Based Models

Transformer-based models first came to light by the work carried out in (Vaswani et al., 2017). The paper demonstrated that the transformer-based architecture depended largely on attention processes for sequences to be processed without requiring sequential dependencies (Illangarathne, Jayasinghe, and Lima, 2024). This represented a major shift in natural language processing which started using self-attentional procedures instead of traditional sequential processing methods (Illangarathne, Jayasinghe, and Lima, 2024). This novel approach can be applied to deepfake detection algorithms as well for better efficiency which shall be explored in this section.

The work in (Soudy et al., 2024) used convolutional vision transformers (ViT) together with CNN for deepfake detection. The input image is separated into visual tokens and image patches in the ViT model. These visual cues are incorporated into fixed-dimensional encoded vectors which are coupled with the position data of every patch. This composite representation is processed by the transformer encoder network similarly to text inputs (Soudy et al., 2024).

2.5 Computational Complexity and Efficiency of Deepfake Detection

Having identified the various mechanisms used for deepfake detection, it is now important to understand the computational complexity and efficiency of these techniques used. It is crucial to have a foundational understanding to be able to assess the efficiency of the deepfake detection models thoroughly. The computational complexity theory can be defined as an examination of the time, memory and other resources needed to solve computational tasks (Sipser, 2013). Time and space are considered essential factors when comparing the performance of various algorithms and deciding which one is better to solve a specific problem (Phalke, Vaidya, and Metkar, 2022). To have a better grasp on this, time complexity and the Big-O notation shall be explained. The amount of time taken by an algorithm to execute is referred to as time complexity which is measured by the number of operations it performs in relation to the input size generally calculated mathematically or theoretically. The maximum time which can be taken by an algorithm for a given input is characterised as the Big O notation or Big O complexity. (Phalke, Vaidya, and Metkar, 2022). The time and space complexity of selected algorithms will be used to determine the efficiency of the existing deepfake detection mechanisms.

2.6 Related Works

- **Comparative Analysis and Evaluation of CNN Models for Deepfake Detection (Ritter, Lucian, Chowanda, et al., 2023)**

This work carried out a comparative analysis of CNN models for deepfake detection. They were assessed based on training accuracy, average loss and testing accuracy using a dataset from FaceForensics++. There were four models tested namely ResNet-152, Mobilenet V3, Convnext Large and EfficientNetB7. There was also LSTM which was added in the architecture of the mechanisms so as to cater for sequential information as well. The primary goal of this research was to evaluate the accu-

racy of the algorithms mentioned to discover different types of deepfakes. For the experiment phase, the mechanisms were imported from existing Pytorch libraries, then proceeding to the collection and analysis part where the dataset of FaceForensics++ was used followed by the same preprocessing process for all the models and finally achieving the result. The F1-Score was also calculated for each of the models. This work concluded that the best CNN-based model for detecting deepfakes is EfficientNetB7 given that it achieved the highest testing accuracy. The research, however, recognises that there can be novel techniques such as attention mechanisms and data augmentation which can be used for better detection of deepfakes. This work focused on testing the accuracy of the mechanisms used but left out other factors such as time and space taken for computation.

- **A Study on Deep Fake Face Detection Techniques (Arya, Goyal, Chawla, et al., 2024)**

This paper carried out an evaluation of various deepfake detection models including neural networks and machine learning methods. Other techniques were also studied such as image analysis, audio analysis and behaviour analysis. These were analysed on the basis of the methodologies used, the key features and the benefits and disadvantages of each method. The work acknowledges that the existing techniques are not able to defend effectively against adversarial attacks such as temporal consistency attacks, zero-day attacks, transfer attacks and many more and thus recognises the need for stronger methods able to detect these. The work also makes mention of a lack of data which can prevent deepfakes from being detected. A part of the paper also did a summary of the breakthroughs in the methods for deepfake detections. These included capsule networks, explainable AI models, cross-model fusion techniques, ensemble learning approaches and real-time processing enhancements. The authors proposed features that could be implemented to improve the existing solutions and these included robustness to zero-day attacks, real-time processing optimisation, hybrid model architectures and cross-domain collaborations. The ethical considerations and how the techniques are viewed by the user was also taken into account in this work. It listed features such as trust in detection systems, ethical frameworks for deepfake detections, public awareness and education and human rights considerations in this section. The paper proposed future research in how deepfake detection algorithms can be improved using blockchain and explainable AI.

- **Comparison of deepfake detection techniques through deep learning (Taeb and Chi, 2022)**

The authors in this work focused on how effective deep learning algorithms are at differentiating fake images against real ones. The algorithms evaluated were Custom CNN, VGG19 and DenseNet-121 using data augmentation in the datasets as well so as to improve performance and make use of less computational resources. The paper also did a comparison of the deepfake datasets publicly accessible outlining which techniques are used for generating the test data. It also included the limitations of the datasets used and how these could be addressed. The techniques studied in this paper were evaluated in terms of accuracy, precision, recall, F1-score, average precision and area under the ROC curve. The work recognised that deep learning algorithms have the ability to differentiate between authentic and fraudulent photographs which is dependable and accurate. The authors concluded that the classifier VGG-19 has better performance as high as 95% when compared to other models in terms of accuracy, F1-score and precision.

- **Deepfake Detection Using Machine Learning Algorithms (Rana, Murali, and Sung, 2021)**

This work argued how traditional machine learning (ML) algorithms perform better than novel deep learning (DL) techniques outlining that ML models have clearer understandability and interpretability with less computational cost. The steps on which the ML mechanism is based include feature development and selection, training, tuning and finally testing the ML classifier. The paper achieved results of above 99% on the datasets tested for the ML models in terms of accuracy and that the speed of getting results is faster than DL algorithms. The datasets used were FaceForensics++, DFDC, Celeb-DF and a newly proposed one by the authors named VDFD. The training phase was carried out through the use of support vector machines, decision tree, random forest, extremely randomised trees, K-Nearest Neighbour, Stochastic gradient boosting and multilayer perceptron network. The algorithms were tested in terms of the Q-test and F-test which allowed for a null hypothesis test. The paper concluded that the computational cost were higher for DL methods and they proposed a ML model for deepfake detection which they argued was more effective. The future works in this paper suggested were to develop better detection mechanisms on the basis of the model put forward.

- **Advanced Deepfake Detection using Machine Learning Algorithms: A Statistical Analysis and Performance Comparison (Rana and Sung, 2024)**

The authors in this paper studied how classical ML methods for detecting deepfakes can be more effective than DL ones. It was once again argued that ML methods had better understandability and interpretability together with lower computation cost. For the experimental part of the project, the datasets used were FaceForensics++, DFDC, Celeb-DF and VDFD where there were preprocessing and analysis methods applied to 400 videos so as to make the ML models better. The feature engineering methods used for analysis were Haralick Texture Features, Histogram of Oriented Gradients, Colour Histogram and Hu Moments. These are the engineering methods or descriptors which were used to create a feature set in the paper. The work employed statistical analysis such as Analysis of Variance (ANOVA) to compare the performance of the various ML algorithms. They were evaluated in terms of precision, recall, F1-Score and accuracy. The experiments in the work had results of around 99% for the ML methods tested on all the datasets in the paper and this consistency of this accuracy was demonstrated through the statistical analysis of the different datasets. The future works in this work goes in the direction of how other types of deepfake media such as audio can be detected and as well as fake media created using generative AI.

- **Investigating the Effectiveness of Deep Learning Approaches for Deep Fake Detection (Berrahal et al., 2023)**

The aim of this paper was to create a CNN-based algorithm for better deepfake detection. To achieve this, the authors first did a comparison of the effectiveness of the existing various deepfake detection mechanisms. The datasets used for training the models included authentic data from CelebA-HQ and fake data as well which was created from a trained generative adversarial network (GAN) based generator. All the evaluated techniques achieved good results but the ResNet50 performed exceptionally with an accuracy of 99.43%. The method used in this paper for the experimental phase involved combining two datasets namely the CelebA-HQ and the flickr faces HQ (FFHQ) which would serve

as the basis for the research. The models were evaluated in terms of accuracy, precision, recall and F1-Score.

2.7 Comparative Analysis

The table below illustrates how various related works compare against each other in terms of the techniques analysed, the key detection features used in the work, the datasets used followed by observations.

Authors	Techniques	Key Detection Features	Datasets Used	Observations
Ritter, Lucian, Chowanda, et al., 2023	CNN: ResNet-152, MobileNetV3, ConvNeXt Large, EfficientNetB7	A video was split into frames for pre-processing. Their metadata was labelled to detect whether they were fake or not.	FaceForensics++	Focused on CNN-based models and their accuracy, but did not compare with other algorithms or address computational cost. The authors noted overfitting and poor generalisation to unseen data. They also pointed out the difficulty of finding one solution for all deepfake types.
Arya, Goyal, Chawla, et al., 2024	Neural networks, ML, behavioural, image, and audio analysis	Reviewed detection features including pixel-level anomalies, audio signatures, and statistical facial deviations.	N/A	Reviewed deepfake detection literature and proposed ethical tools using Explainable AI and blockchain. Efficiency was not evaluated. Trade-offs between detection types were highlighted, and Capsule Networks noted as computationally heavy.
Taeb and Chi, 2022	CNN (VGG19, DenseNet-121)	Used data augmentation to improve model performance and reduce computational cost.	Augmented real/fake face detection dataset	Compared CNN models, emphasising accuracy and efficiency improvements. Models performed better on GAN-generated images due to artefacts, but some were less efficient computationally.
Rana, Murali, and Sung, 2021	Machine Learning Techniques	Extracted 200 frames from 400 videos, tracked faces, and used 'dlib' for feature extraction before classification.	FaceForensics++, DFDC, Celeb-DF, VDFD	Argued ML methods are more interpretable and cost-effective than DL, and proposed a viable ML-based model. Classical ML was shown to be more efficient than DL in several cases.
Rana and Sung, 2024	Machine Learning Methods	Used 'dlib' to extract features for classifiers to identify fake faces.	FaceForensics++, DFDC, Celeb-DF, VDFD	Performed ANOVA and statistical comparisons of ML vs DL. ML approaches often required fewer resources while achieving competitive performance.
Berrahal et al., 2023	CNN	Merged data from different datasets to assess algorithmic performance.	CelebA-HQ, FFHQ	Proposed a CNN classifier trained on GAN-generated data. Achieved 99.43% accuracy using ResNet50. Some images with high noise or manipulation could not be predicted correctly.
Akhtar, 2023	Deep Learning Techniques	Surveyed detection methods for identity swap, attribute manipulation, face reenactment, and full face synthesis.	FaceForensics++, CelebA-HQ, FFHQ, VoxCeleb, DFDC	Provided a detailed survey of generation/detection methods. Highlighted that more research is still needed for a universal, effective detection solution.

Table 2.1: Comparative analysis of related works

2.8 Research Gap

This section contains the main observations from existing related works and also outlines which parts of the topic have not been addressed extensively or not at all. There are various works which evaluated the performance of deep learning based deepfake detection methods but the focus was mainly on only one type of deep learning algorithm covering mainly CNN. There are many research which attempted to combine different neural networks such as CNN with RNN to explore if the method was more effective in discovering deepfakes. There are very few papers which explored how transformer-based algorithms perform in detecting fake images which are based on self-attentional methods and not conventional sequential processing methods. Some interesting works even used data augmentation for models to be able to perform better with lower computation cost and also generated their own fake images using a GAN-based generator. There are authors who compared machine learning methods against deep learning one arguing that ML ones perform better following results from their experiments and they also performed statistical analysis of these methods to support their claim. Most of the existing work measure accuracy in terms of precision, accuracy, recall and F1-score.

However, following the analysis of the various related works, it is observed that very few of the works covered the computational complexity of the algorithms used for deepfake detection. The present paper aims at addressing this area by comparing different algorithms based on deep learning methods such as CNN, RNN and transformer-based through time and space complexity, Big-O notation and other measures of computational efficiency.

Chapter 3

Methodology

3.1 Research Design and Approach

The purpose of this section is to provide an outline of the research design, the experimental setup and data collection which will serve to address the research questions. For instance, for comparing the different algorithms such as CNNs, RNNs and transformers in terms of computational complexity, accuracy and real-world applicability and as well as the trade-off between computational efficiency and real-world applicability.

This research will consist of both a theoretical, experimental approach so as to perform a comparative analysis of the algorithms. The study will involve the selection and implementation of deep learning models such as CNN, RNN and transformer architectures which will then be evaluated based on existing deepfake datasets. A quantitative measurement will also be carried out in terms of accuracy and the main computational efficiency metrics which will be elaborated in this section followed by the comparative analysis of the obtained results. The next part dives into the subject as to why the specific deep learning algorithms for deepfake detection were chosen.

3.2 Algorithm Selection

A systematic selection of deep learning deepfake detection models is essential to be able to conduct a meaningful analysis which will serve to address the research questions related to comparative computational complexity, accuracy and real-world applicability. This section outlines on what basis specific algorithms were chosen for CNN, RNN and transformer-based models which be the subjects of this study. The aim is to choose architectures which are popular in literature, have various computational characteristics and which have a comprehensive basis for evaluating efficiency of the different selected neural networks. This part will focus on why particular models were chosen and also explain why they were included in the study.

The paper (Yasser et al., 2023) did a thorough analysis of the deepfake detection models based on CNN such as EfficientNet-B4 and XceptionNet. Based on the results obtained in the paper, it will be explained why XceptionNet was chosen for measuring the efficiency of CNN architectures in this study. XceptionNet is a neural network architecture based model. It operates on the basis of depthwise separable convolutions where the channel-wise and spatial convolutions are divided. This contributes to lower costs of computa-

tion while keeping a high level of accuracy (Yasser et al., 2023). In order for the authors in the paper to come to the conclusion that XceptionNet has several benefits over EfficientNet-B4, experiments were conducted using the datasets Celeb-DF and FaceForensics++ separately and even by combining them. The evaluation metrics focused on the Area Under Curve (AUC) and the log loss metrics. The AUC demonstrates the relationship between the binary classifiers for the true positive and false positive rates whereas log loss is used to measure the effectiveness of the models. XceptionNet achieved a score of 90.15% for the AUC and a result of 0.3177 for log loss compared to 92.99% for the AUC and a log loss of 0.2719 for EfficientNet-B4 based on the results achieved for the combined datasets. Even though, it is seen that the EfficientNet-B4 has achieved better results, based on the paper being analysed, XceptionNet is more favourable to be chosen for the comparing computational complexity because of its ability to maintain a high level of accuracy with low computational costs.

The model which would be selected for the study of the computational complexity of RNN-based algorithms shall be discussed in this part. After research, it seemed to be better to analyse a hybrid model for the RNN study as it would be more representative of the solutions being used. The paper (Tipper, Atlam, and Lallie, 2024) claims that there is a near-perfect accuracy in deepfake detection when using a model based on CNN and LSTM (which is a type of RNN). The work provided a summary of existing research which used CNN and CNN-LSTM for deepfake detection where it outlined the performance of the models achieved in each work, the metrics and datasets used for evaluation and frame rates where available. It was found that most models which used LSTM together with CNN performed better than those which used CNN only based on the results obtained using various existing works. The better performance is the result of the effectiveness of CNNs in extracting spatial characteristics from video frames including textures, facial structures and lighting. On the other hand, LSTM models capture temporal dependencies by analysing patterns across consecutive frames. The combination of the models thus enables the exploitation of both spatial and temporal features resulting in improved performance. For instance, an accuracy of 94.29% was achieved for models using CNN-LSTM whereas the highest accuracy for a CNN-only model was 91.72%. It was equally observed that the training time was faster and there was less loss times when using CNN-LSTM when compared to CNN-only algorithms. These are the main reasons which contributed to the selection of a hybrid model based on CNN and LSTM for comparing the efficiency of RNN based algorithms for this study.

For the transformer-based algorithms, the vision transformer (ViT) models will be studied. This was motivated mainly by the results obtained in the work carried out in the paper (Ghita et al., 2024) which outlined relatively high results of 89.9125% for ViT models. For the evaluation phase, the training and testing was carried out using the Kaggle image dataset where there were both real and fake images. The metrics used for the evaluation was precision, recall and F1-score. The model achieved an accuracy of 0.90 for the training dataset used but this number was relatively low for the validation dataset, that is, 0.75. It was also noted that larger datasets are needed for the training of ViT models and if smaller datasets are being used, they should be fine-tuned first. Based on available information and existing research, ViT will be studied to measure the computational complexity of the transformer-based models for deepfake detection.

The table below provides a summary of the specific algorithms which will be dealt with in this work along with brief explanations.

Model	Type	Key Features
XceptionNet	Convolutional Neural Network (CNN)	Depthwise separable convolutions which allow for high accuracy with low computation costs.
XceptionNet + LSTM	Hybrid CNN-RNN	Combines both spatial and temporal analysis and has high accuracy rates.
Vision Transformer (ViT)	Transformer-based	Self-attentional features and used largely for NLP tasks.

Table 3.1: Algorithm Selection

The above table provided a concise explanation of why the particular models were chosen for this study for a clearer view. The next section of this report will contain the process behind the datasets chosen and how the experiments were carried out to measure the computational complexity of the various algorithms.

3.3 Dataset Selection and Preprocessing

There are two different datasets which were selected for the experiments in this work namely Celeb-DF (v2) from github and the FaceForensics++ dataset (Rössler et al., 2019). The Celeb-DF dataset is made up of 590 real videos and 5639 deepfake ones selected from YouTube videos which are publicly available. The videos are based on the interviews of 59 celebrities and represent various ethnic groups, genders and ages. The frame rate for the videos is at 30 frames per second and the average length of the videos are 13 seconds (Yuezun Li et al., 2020). The dataset has more realistic deepfake videos which allow for a significant effect on analysis (Yasser et al., 2023) as they were created using an improved deepfake synthesis algorithm which helped enhance visual quality of the images (Yuezun Li et al., 2020).

The FaceForensics++ dataset consists of 1000 original video sequences which have been modified with different types of automated face manipulation methods namely Face2Face, FaceSwap, DeepFakes and NeuralTextures (Rössler et al., 2019).

These features of the datasets enable better detection accuracy and also makes it more reliable to measure the computational efficiency of the algorithms. The data was divided into 70% for training and 30% split equally between testing (15%) and validation (15%).

3.3.1 Preprocessing for XceptionNet

The Xception architecture is pre-trained on ImageNet and thus, there are particular preprocessing steps which were applied to all input images. First, a fixed number of frames were extracted from the video files and each frame was resized to 299x299 pixels. The function `keras.applications.xception.preprocess_input` was first called on the input images before passing them to the `xception.preprocess_input` model so that the pixels can be scaled between -1 and 1 (Chollet, 2017). Finally, the extracted frames were organised into folders, 'real' and 'fake' to allow the Keras's ImageDataGenerator to automatically assign labels based on the directory structure.

3.3.2 Preprocessing for XceptionNet-LSTM

The preprocessing for the CNN part of this one will be the same as XceptionNet architecture as the same model will be used for the CNN part of the CNN-LSTM model. Thus, a fixed number of frames would be extracted from the videos and resized to 299x299 pixels. The frames are then grouped into temporal sequences where each sequence represents a video. These sequences were then either padded or truncated to a fixed length. This preprocessing allowed for both spatial and temporal data to be captured for testing.

3.4 Evaluation Metrics

To assess the performance of the algorithms, both detection accuracy and computational efficiency were evaluated. There are standard classification metrics which were used such as accuracy, precision, recall and F1-score to measure the detection performance on the Celeb-DF and FaceForensics++ datasets. The formulae for the metrics mentioned are outlined below which were based on those provided in the paper (Tipper, Atlam, and Lallie, 2024).

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = 2 * \frac{Precision * Recall}{Precision + Recall}$$

Where,

Accuracy	The number of correct predictions made out of the total predictions.
Precision	Shows how reliable the model is in detecting real and fake videos.
Recall	The percentage of videos correctly detected as fake.
F1	Combines precision and recall to illustrate how the model can correctly detect both real and fake videos.

Table 3.2: Evaluation Terms Definition (Taeb and Chi, 2022)

The table above explains the terms that would be used to evaluate the performance of the models. These would be explained clearer in the results section when applied for analysis. Table 3.3 explains the meaning of the variables used in the equations in a simple way.

On the computational side, metrics such as training and inference time, memory usage and the operation count will be used.

True Positives (TP)	Fake videos correctly predicted as fake.
True Negatives (TN)	Real videos correctly predicted as real.
False Positives (FP)	Real videos wrongly predicted as fake.
False Negatives (FN)	Fake videos wrongly predicted as real.

Table 3.3: Evaluation Variables Definition

Where,

Metric	Description
Training time	Time taken to train the model on the full dataset per epoch. This was used to indicate overall training efficiency.
Inference time	Time taken to make a prediction per frame. Showed how quickly the model can make predictions.
False Positives (FP)	Incorrectly classified real inputs as fake. Shows detection reliability and accuracy.
Memory usage	Memory used during training (RAM/VRAM). Used to assess resource demands during training.
Big-O (Time and space complexity)	Theoretical estimation of scalability with input size. To analyse scalability as dataset size, epoch or model size increase.
FLOPs (Floating point operations)	Total number of arithmetic operations needed to process one input (forward pass). Provided hardware-independent measure of computational load.

Table 3.4: Computational Efficiency Evaluation Metrics

The table above provides a summary of the metrics which would be used to determine the computational efficiency of the various algorithms used for deepfake detection. The next chapter lays out and analyses the results obtained following the experiments conducted. The operation count (per forward pass, FLOPs) can be calculated by using the following formula for the CNN model.

$$OperationCount(CNN-Model) = (H_{out} \cdot W_{out} \cdot K^2 \cdot C_{in} \cdot C_{out})$$

- H_{out}, W_{out} : height and width in output feature map
- C_{in} : number of input channels
- C_{out} : number of output channels
- K^2 : kernel height and width

For the hybrid model using CNN and RNN, the operation count can be derived by the following formula.

$$OperationCount(\text{Hybrid-Model}) = 20 \cdot (H_{\text{out}} \cdot W_{\text{out}} \cdot K^2 \cdot C_{\text{in}} \cdot C_{\text{out}}) + 20 \cdot N_{\text{units}} \cdot (N_{\text{units}} + N_{\text{inputs}})$$

where:

$H_{\text{out}}, W_{\text{out}}$ Output height and width of CNN layer

K Kernel size of convolutional layer

$C_{\text{in}}, C_{\text{out}}$ Number of input and output channels of CNN layer

N_{units} Number of LSTM units

N_{inputs} Number of input features to LSTM

20 Number of frames (timesteps) processed by TimeDistributed CNN/LSTM

The formulae for the operation count were adapted from (Holleman, 2018) after the experiments were carried out and the results were saved which explains the choice of the layers and number of frames and other variables in the equations as well.

Chapter 4

Experiments and Results

4.1 Experimental Setup

The experiments were carried out on Google Colab Pro due to the high performance GPUs that it offers such as A100 GPU, L4 GPU and TPUs. Google Colab also provides a modular approach for the code where each part of the code can be tested in individual cells which is also one of the reasons which motivated its use. On the software side, the operating system used is Windows 10 and the programming language used is Python 3. The frameworks used are TensorFlow for the deep learning and the main libraries used include OpenCV, Numpy, Pandas.

4.2 Deepfake Detection Performance

4.2.1 CNN-based Algorithms

For the model developed using only CNN, the evaluation metrics can be summarised in the table below.

	CNN	
	Celeb-DF-V2	FF++
Accuracy	85%	93%
Precision	87%	91%
Recall	67%	93%
F1	76%	92%

Table 4.1: Performance comparison of CNN on Celeb-DF and FF++ datasets

The figures in table 4.1 represent the performance of the CNN algorithm (XceptionNet) in detecting deepfakes on the Celeb-DF-v2 dataset and the FaceForensics++ dataset. For the Celeb-DF-v2 dataset, it can be seen that after the model training, it achieved a relatively high rate for the performance. On the other hand, it is seen that the results on the FaceForensics++ dataset are significantly better as all of the metrics are over 90%. It is to be noted that the dataset was split into 70% for training and 15% for both training and validation so as to avoid overfitting.

4.2.2 Hybrid Algorithms

For the model developed using both CNN and RNN (LSTM) together, the evaluation metrics can be summarised in the table below.

	CNN-LSTM	
	Celeb-DF-V2	FF++
Accuracy	75%	95%
Precision	66%	98%
Recall	59%	91%
F1	63%	94%

Table 4.2: Performance comparison of CNN-LSTM on Celeb-DF and FF++ datasets

For the Celeb-DF-v2 dataset, table 4.2 shows the results obtained through the experiments based on the model developed using both CNN and RNN. It is seen that it has relatively high rates of accuracy and precision which are at 81% and 83% respectively. However, for the other metrics such as recall and F1-score, the performance are comparatively lower which are at 59% and 69% respectively. For the FaceForensics++ dataset, the performance is higher with all the metrics being over 90%.

4.3 Computational Complexity

4.3.1 CNN-Based Algorithms

This section focuses on time and space (memory) taken by the algorithms to complete the detection of the deepfakes. This can be measured in terms of training time, inference time, number of false positives, memory usage and the operation count per forward pass. The following table summarises the results obtained for the CNN model on the two datasets used.

	CNN	
	Celeb-DF-V2	FF++
Training Time	9124 seconds	45684 seconds
Inference Time	1141 seconds	782 seconds
False Positives	103	954
Memory Usage	17 GB	67 GB
Operation Count (per forward pass, FLOPs)	7.99×10^9	7.99×10^9

Table 4.3: Computational Complexities of CNN

From table 4.3, it is noted that it is less computationally intensive as compared to experiments carried out on the FaceForensics++ dataset. However, the number of false positives is lower for the CELEB-DF-v2 dataset. These efficiency metrics would be interpreted in more detail in the next chapter.

4.3.2 Hybrid Algorithms

For the model trained on both RNN and CNN, the computational complexities are summarised in the table below.

	CNN-LSTM	
	Celeb-DF-V2	FF++
Training Time	8381 seconds	65936 seconds
Inference Time	964 seconds	1170 seconds
False Positives	5	25
Memory Usage	5 GB	424 GB
Operation Count (per forward pass, FLOPs)	2.23×10^7	2.23×10^7

Table 4.4: Computational Complexities of CNN-LSTM

The table 4.4 shows the computational complexities of the hybrid algorithms using CNN and LSTM on both the datasets used in the experiments.

Chapter 5

Findings and Discussion

The aim of this chapter is to provide a comprehensive review of the findings and to discuss their implications. The results related to model accuracy and computational efficiency will be interpreted thus addressing the core research questions of this study while acknowledging the limitations encountered during the project as well.

5.1 Analysis of Experimental Findings

5.1.1 Performance Analysis

This section focuses on interpreting the results obtained from experiments for the performance of the models in the previous chapter and discussing their implications as well. The **CNN model** using XceptionNet trained on the **Celeb-DF-v2** dataset achieved a relatively high performance illustrated by the high values of the metrics obtained from experiments. For accuracy, it had an average score of 85% implying that the number of correct predictions were right 85% of the time. The precision for the same model was at 87% which indicates that it is highly reliable in detecting both real and fake videos correctly. The recall was at 67% which means that 67% of the videos were correctly detected as fake by the model. The F1-Score which is a combination of precision and recall was at 76% which illustrates the ability of the model to correctly predict both real and fake videos. On the other hand the **hybrid model** using LSTM and XceptionNet trained on **Celeb-DF-v2** dataset had comparatively lower performances in some areas. The accuracy and precision results for this model were at 75% and 66% respectively. This shows that it could correctly distinguish between real and fake videos 75% of the time but its precision value shows that the model may not always be reliable in detecting fake videos. However, the recall and F1 scores were significantly lower compared to the CNN model. The recall was at 59% indicating that there was a relatively low percentage of videos being correctly classified as fake. The F1 score was at 63% implying that the model could be relied upon only around 63% of the time.

The accuracy of the CNN model on the Celeb-DF-v2 dataset can be illustrated graphically to show how it performed over the training and validation phases during the process.

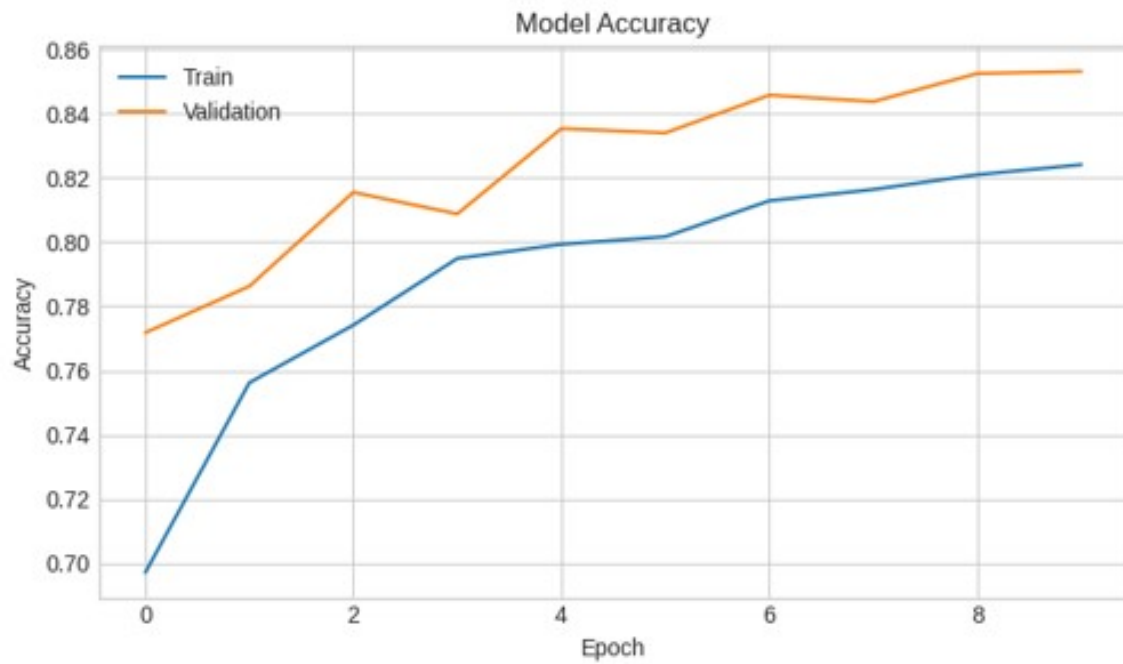


Figure 5.1: CNN Model Performance Over Epochs for Celeb-DF Dataset

From the figure above, it is seen that the CNN model was relatively consistent in terms of accuracy over the validation and training phases in terms of accuracy. The results which were achieved implied that the predictions were correct on an average of 89% of the time including detecting real and fake videos correctly when using the Celeb-DF-v2 dataset.

The results in terms of performance for accuracy for the hybrid model using CNN and LSTM can be illustrated as follows for the Celeb-DF-v2 dataset.

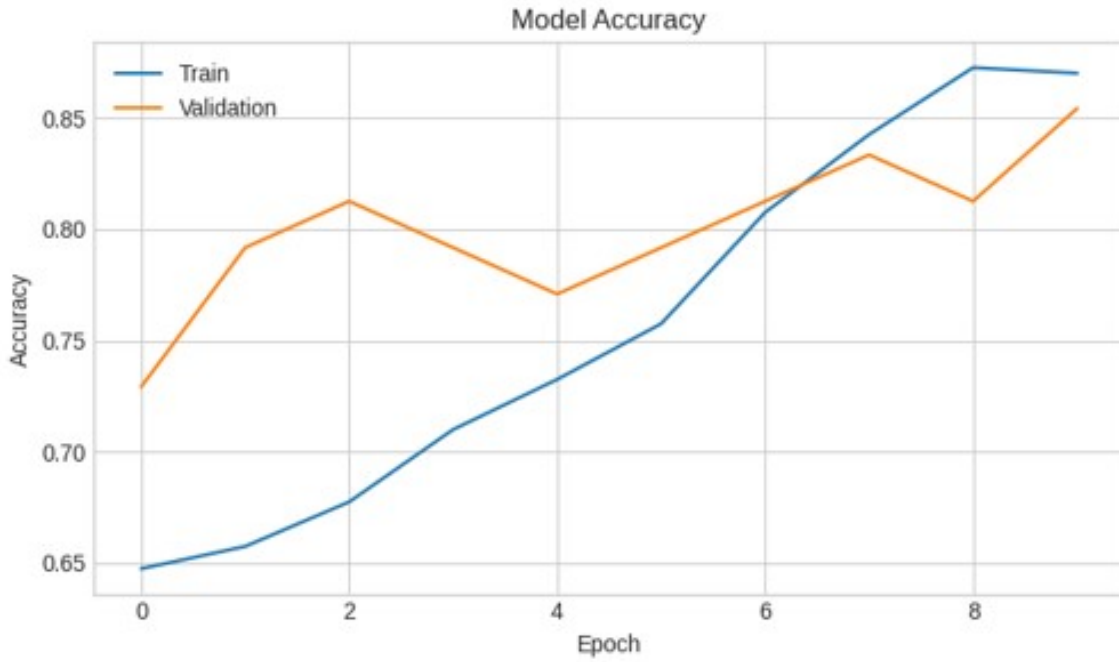


Figure 5.2: Hybrid Model Performance Over Epochs for Celeb-DF Dataset

The graph above illustrates the performance in terms of accuracy of the hybrid model over epochs in terms of accuracy for the Celeb-DF-v2 dataset.

The **CNN model** trained on the **FaceForensics++** dataset achieved considerably high performance values with all of them being over 90%. The score 93% of accuracy shows its robust ability to correctly differentiate between real and fake videos and its high reliability as well is illustrated by the high result of 91%. A large proportion of the fake videos were detected which is confirmed by the result of 93%. 92% of the videos was correctly identified as real or fake which is shown by the recall value. The accuracy for this model on the FaceForensics++ dataset can be illustrated below.

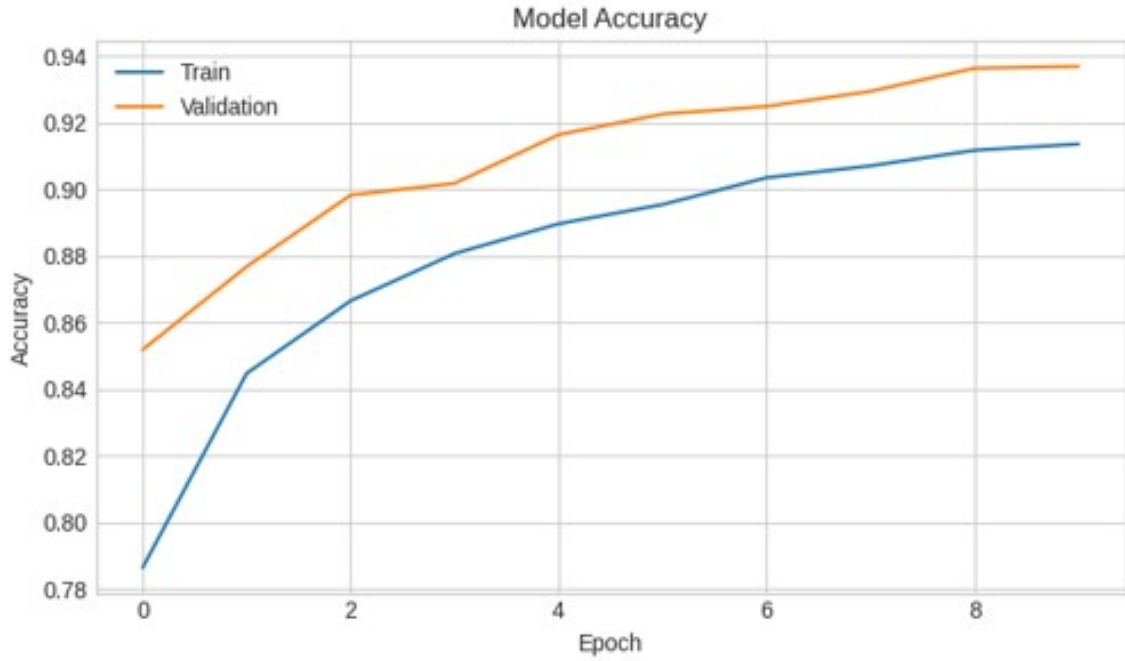


Figure 5.3: CNN Model Performance Over Epochs for FaceForensics++ Dataset

Figure 5.3 shows how the accuracy of the CNN model evolved over the training and validation phase on the FaceForensics++ dataset.

From the results obtained, it can also be observed that the CNN model outperformed on the Celeb-DF-v2 dataset but not the FaceForensics++ one. The possible explanations for this could include the fact that FaceForensics++ uses multiple algorithms to manipulate videos such as DeepFakes, Face2Face, FaceSwap and NeuralTextures. This manipulation introduce strong visual features which might make it harder for the CNN model to effectively detect the deepfakes. On the other hand, the Celeb-DF-v2 dataset has more diversity in factors such as faces and lighting. The texture inconsistencies can thus be exploited by the CNN model for it to achieve a better performance.

The **hybrid model** trained on the **FaceForensics++** dataset also had significantly high scores in terms of performance, where all the metrics were over 90%. For accuracy, it achieved an average score of 95% implying that the model correctly differentiated between real and fake videos 95% of the time. The precision was at an average score of 98% which shows the high reliability of the model in correctly detecting if the videos are deepfakes or not. The correct percentage of videos detected as fake was at 91% illustrated by the recall value. The F1-score at 94% indicates a high proportion of the videos are correctly classified in terms of real and fake. The accuracy of the hybrid model on the FaceForensics++ dataset can be illustrated as follows.



Figure 5.4: Hybrid Model Performance Over Epochs for FaceForensics++ Dataset

Figure 5.4 above shows the results obtained during training over 10 epochs and how it progressively went up to achieve results of more than 90%.

The analysis above shows that the performance of the CNN model is better than the hybrid one on the Celeb-DF-v2 dataset. However, the hybrid algorithm seems to perform slightly better than the CNN one on the FaceForensics++ dataset.

5.1.2 Efficiency Analysis

In terms of efficiency, the **CNN model** proved to be more resource intensive than the hybrid model on the **Celeb-DF-v2** dataset. The training time was at 9124 seconds with inference time of 1141 seconds implying that it took 1141 seconds to make a prediction per frame. The number of false positives were at 103 and the total memory used during the training was 17 GB. For the **FaceForensics++** dataset, the **CNN model** had a training time of 45684 seconds, a total memory usage of 67 GB and it had 954 videos wrongly detected as fake. However, these high values are due to the fact that the dataset is much larger than the Celeb-DF-v2 dataset as it contains 1000 videos manipulated using several techniques. Nevertheless, the inference time was still lower at 782 seconds which implies that it took less time to make a prediction per frame as compared on

the Celeb-DF-v2 dataset which was at 1141 seconds.

On the other hand, the algorithm based on **both CNN and RNN** architectures on the **Celeb-DF-v2** dataset proved to be more efficient from the value of the metrics obtained used to measure the computational complexity when compared to the CNN-based model. The training time was 8381 seconds with inference time of only 964 seconds which is the time taken to make a prediction per frame. The amount of false positives were only 5 which means that only 5 real videos were wrongly flagged as fake on the whole dataset. The total memory usage during the training, testing and validation phase by the script was 5 GB. For the **FaceForensics++** dataset, the **hybrid model** proved to be more computationally intensive which performed better in terms of performance. The total training time was at 65936 seconds with a total memory usage of 424 GB. The inference time was at 1170 seconds which shows a high time taken to make predictions per frame but however, only 25 false positives were recorded implying that it was highly accurate in detecting deepfakes.

It was also noted that the memory usage for the hybrid model using CNN-LSTM was really high during the experiments. This is due to the fact that the model handles both the spatial and temporal features at once as compared to the CNN model which extracts each frame of a video. The memory usage of the hybrid model is therefore high because it requires the entire sequence of the features which the CNN model extract in terms of large feature maps per frame.

5.1.3 Trade-Off between Performance and Efficiency for Deepfake Detection Algorithms

This study demonstrates that there is a trade-off between performance and efficiency of both the algorithms based on CNN and the CNN-LSTM one from the results obtained in the experiments performed. It is seen that the experiments performed on the FaceForensics++ dataset have extremely high performance of more than 90% for all the metrics measured. However, it is noted that both the algorithms are computationally resource intensive with total memory usage of up to 424 GB for the hybrid model. For the Celeb-DF-v2 dataset, it is seen that performance is relatively lower but computation power is lower as well. This may be due to the fact that the dataset is smaller than the FaceForensics++ dataset but it still has 5639 deepfake videos and 590 real ones. The memory used during the whole training was as low as 5 GB for the hybrid model and even the training time was at only 8381 seconds. The choice for deciding the use of an algorithm with relation to performance or efficiency will depend on the type of organisation using it. For instance, the requirements for a private company would be different from that of a national security system for the government of a country. For example, the trade-off between model complexity and computational cost can be a determining factor for deployment in real-time scenarios such as social media moderation as lightweight models with slightly lower accuracy may be preferred but with faster inference time. Another example might be the case for law enforcement agencies who would normally accept slower and more resource-intensive models for higher reliability. For example, they might prefer the CNN-LSTM model which was the least efficient one on the FaceForensics++ dataset but it was the one which achieved the highest accuracy of 95%.

5.2 Addressing Research Questions

- **How do CNNs-based and RNNs-based deepfake detection models compare in terms of computational complexity and accuracy?**

This study carried out experiments to determine the performance of algorithms using CNN and RNN-based ones as well in which the accuracy was also measured. In terms of accuracy for on the Celeb-DF-v2 dataset, it is noted that the accuracy is higher, that is, at 85% and it is only at 75% for the hybrid model using CNN and LSTM. Nevertheless, the algorithms perform significantly better on the FaceForensics++ dataset with the CNN model achieving a high accuracy score of 93% and the hybrid one achieving a score of 95% implying they correctly detected a high percentage of videos on the FaceForensics++ dataset. On the computational complexity side, it is seen that both the models are more resource intensive when working with the FaceForensics++ dataset with training times going up to 65936 seconds and total memory usage up to 424 GB for the hybrid model and up to 45684 seconds and total memory usage up to 67 GB for the CNN model. It is seen that the computational efficiency of both the algorithms have different behaviours with the main factors affecting the results being the dataset used.

- **To what extent do time and space complexities affect the efficiency of each algorithm?**

Following the experiments in this project, it was seen that the effects of time and space complexities do not have a linear relationship. There were distinct behaviours for both the algorithms depending on the dataset on which the experiments were performed as well. It was, however, seen that on the Celeb-DF-v2 dataset, there were less false positives detected with training times being lower than experiments carried out on the other dataset. For instance, for the CNN model, the training time was at 9124 seconds and memory used amounted to 17 GB and 103 false positives were detected compared to 954 false positives detected on the FaceForensics++ dataset where training time was 456840 seconds and memory usage was at 67 GB. Similarly for the hybrid model, on the Celeb-DF-v2 dataset the number of false positives were only 5 where training time was 8381 seconds and memory used amounted to 5 GB compared to 25 false positives detected on the FaceForensics++ dataset where training time was 65936 seconds and memory used was 424 GB. A detailed analysis of the efficiency of the algorithms was given above in the section ‘Efficiency Analysis’.

- **Which algorithm offers the best trade-off between computational efficiency and applicability in real-world applications, and why?**

The section ‘Trade-Off between Performance and Efficiency for Deepfake Detection Algorithms’ covered part of this research question which illustrated how a sacrifice had to be made in either performance (for example, in terms of accuracy) or in efficiency (for example, in terms of training time and memory usage). For the applicability in real-world scenarios, it depends on the requirements as to why deepfakes have to be detected. For instance, a private organisation would not be needing them for the same reasons as a national security service of a country. Thus, the decision on what to sacrifice would depend on the needs of the organisation using the algorithms for deepfake detection.

5.3 Discussions and Limitations of the Study

The main setbacks in this project were encountered during the experiment phase to determine the efficiency of various algorithms such as XceptionNet for CNN-based and a combination of XceptionNet and LSTM which is a hybrid model based on both CNN and RNN. The issue was the large dataset sizes which made the preprocessing and training times extremely long as a result of which the experiments were carried out on Google Colab Pro instead of Jupyter Notebook. This is because Google Colab Pro provides access to more powerful virtual processors. However, the processes were still time consuming. Following this, the decision to use multithreading in scripts and as well as multiple workers simultaneously was taken in an attempt to reduce preprocessing duration of dataset and training duration of models. After this measure was taken, the processes were still time consuming but had clearly reduced considerably and thus explaining the decision to stick with this approach.

The FaceForensics++ dataset was much larger than the Celeb-DF-v2 dataset due to which it had to be preprocessed in multiple batches simultaneously to speed up processing times and avoid errors. After this step, all the batches preprocessed were brought together in a single folder named 'ff_unified' with 'real' and 'fake' folders inside. However, after this a Google Drive timeout error was being encountered with the root cause the size of the dataset. To solve this, an attempt was made to use Low-Rank Adaptation of Large Language Models (LoRA) (Hu et al., 2022) but it was still unsuccessful. A brief description of LoRA could be put as a method where the numbers of trainable parameters are brought down by learning pairs of rank-decomposition matrices while freezing the original weights for training (Hu et al., 2022). At last, the files into the 'ff_unified' folders were further subdivided into subfolders to avoid the timeout issue following which, the scripts performed as intended. These limitations contributed to the fact that algorithms based on vision transformers could not be analysed also taking into considerations the time constraints of the project.

The results in this work did not measure only accuracy but it also demonstrated how efficiency impacts the real-world application of deepfake detection. They highlighted the trade-off between accuracy and computational cost thus showing which models can be deployed in systems such as real-time systems for better performance. The results may also serve as a guide for the deployment of deepfake detection algorithms as it identified which algorithms are better suited for large-scale or resource-constrained environments. It should, however, be noted that deepfake media remains a complex issue which will need algorithms beyond CNN and RNN to solve and the results obtained gives directions about future research to explore other approaches in deepfake detection as well.

Chapter 6

Conclusion and Future Works

6.1 Conclusion

This project carried out experiments to analyse mainly the computational complexity and time and space efficiency of deep learning algorithms for deepfake detection. The algorithms used were a CNN model and a hybrid model based on CNN and LSTM to analyse the behaviour of RNNs as well. The trade-off between efficiency and performance was also studied. The models were evaluated on the Celeb-DF-v2 and FaceForensics++ datasets. However, there were setbacks encountered during the project mainly due to the large sizes of the datasets which made preprocessing of the data more time consuming and more complex. Nevertheless, the experiments could be carried out for both the models and the results for the performance and efficiency as well as the trade-off between the computational efficiency and real-world applicability were recorded and analysed in this study. It was seen that the models performed relatively well on the FaceForensics++ dataset accuracy of 95% achieved by the hybrid model and 93% by the CNN model. It was also noted that the CNN model was less computationally intensive on the FaceForensics++ dataset but it got a high number of false positives which was at 954 and the hybrid model got 25 false positives but it was more computationally intensive. On the other hand, it was observed that the model used less computation power on the Celeb-DF-v2 dataset but their performances were relatively lower with the accuracy of 85% achieved by the CNN model and 75% achieved by the hybrid model.

6.2 Future Works

The scope of this research focused on the analysis of the performance and computational complexity of CNN-based and RNN-based through a combination of CNN and LSTM algorithms for deepfake detection. Further research could include analysing the same issues but using other algorithms such as those based on vision transformers and on different datasets and even other types of data structures as well. For instance, with the high GPU costs observed during the experiments especially for the CNN-LSTM model, future works could explore lighter hybrid models or make use of pruning.

References

- Al-Adwan, Aryaf et al. (2024). "Detection of Deepfake Media Using a Hybrid CNN–RNN Model and Particle Swarm Optimization (PSO) Algorithm". In: *Computers* 13.4, p. 99.
- Akhtar, Zahid (2023). "Deepfakes generation and detection: a short survey". In: *Journal of Imaging* 9.1, p. 18.
- Almars, Abdulqader M (2021). "Deepfakes detection techniques using deep learning: a survey". In: *Journal of Computer and Communications* 9.5, pp. 20–35.
- Anjaneyulu, Battula Prasanna et al. (2024). "DeepFake Detection using Convolutional Neural Networks (CNN) and Recurrent Neural Network (RNN)". In: *2024 5th International Conference on Data Intelligence and Cognitive Informatics (ICDICI)*. IEEE, pp. 711–716.
- Arya, Mudit, Umang Goyal, Simran Chawla, et al. (2024). "A Study on Deep Fake Face Detection Techniques". In: *2024 3rd International Conference on Applied Artificial Intelligence and Computing (ICAAIC)*. IEEE, pp. 459–466.
- Berrahal, Mohammed et al. (2023). "Investigating the effectiveness of deep learning approaches for deep fake detection". In: *Bulletin of Electrical Engineering and Informatics* 12.6, pp. 3853–3860.
- Chollet, François (2017). "Xception: Deep learning with depthwise separable convolutions". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 1251–1258.
- Ghita, Bogdan et al. (2024). "Deepfake image detection using vision transformer models". In: *2024 IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom)*. IEEE, pp. 332–335.
- Güera, David and Edward J Delp (2018). "Deepfake video detection using recurrent neural networks". In: *2018 15th IEEE international conference on advanced video and signal based surveillance (AVSS)*. IEEE, pp. 1–6.
- Hollemans, Matthijs (2018). *How fast is my model? — machinethink.net*. <https://machinethink.net/blog/how-fast-is-my-model/>. [Accessed 25-09-2025].
- Hu, Edward J et al. (2022). "Lora: Low-rank adaptation of large language models." In: *ICLR* 1.2, p. 3.
- Illangarathne, Pooja, Nethari Jayasinghe, and AB Duweeja de Lima (2024). "A Comprehensive Review of Transformer-Based Models: ChatGPT and Bard in Focus". In: *2024 7th International Conference on Artificial Intelligence and Big Data (ICAIBD)*. IEEE, pp. 543–554.
- Jadhav, Sairaje S et al. (2024). "Deep Fake Detection using ResNext Convolutional Neural Network (CNN) combined with a Recurrent Neural Network (RNN)". In: *2024 5th IEEE Global Conference for Advancement in Technology (GCAT)*. IEEE, pp. 1–6.

- Kumar, Koushal and Gour Sundar Mitra Thakur (2012). "Advanced applications of neural networks and artificial intelligence: A review". In: *International journal of information technology and computer science* 4.6, pp. 57–68.
- Li, Yuezun et al. (2020). "Celeb-df: A large-scale challenging dataset for deepfake forensics". In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 3207–3216.
- Phalke, Swapnil, Yogita Vaidya, and Shilpa Metkar (2022). "Big-O time complexity analysis of algorithm". In: *2022 international conference on signal and information processing (IConSIP)*. IEEE, pp. 1–5.
- Rana, Md Shohel, Beddhu Murali, and Andrew H Sung (2021). "Deepfake detection using machine learning algorithms". In: *2021 10th International Congress on Advanced Applied Informatics (IIAI-AAI)*. IEEE, pp. 458–463.
- Rana, Md Shohel, Mohammad Nur Nobi, et al. (2022). "Deepfake detection: A systematic literature review". In: *IEEE access* 10, pp. 25494–25513.
- Rana, Md Shohel and Andrew H Sung (2024). "Advanced Deepfake Detection using Machine Learning Algorithms: A Statistical Analysis and Performance Comparison". In: *2024 7th International Conference on Information and Computer Technologies (ICICT)*. IEEE, pp. 75–81.
- Reddy, Elkapally Mani Sathwik, Aitipamula Pavan Kumar, and Polisetty Swetha (2024). "Deepfake video detection using CNN and RNN with OPTICAL FLOW features". In: *2024 IEEE International Students' Conference on Electrical, Electronics and Computer Science (SCEECS)*. IEEE, pp. 1–7.
- Ritter, Patrick, Devan Lucian, Andry Chowanda, et al. (2023). "Comparative Analysis and Evaluation of CNN Models for Deepfake Detection". In: *2023 4th International Conference on Artificial Intelligence and Data Sciences (AiDAS)*. IEEE, pp. 250–255.
- Rössler, Andreas et al. (2019). "FaceForensics++: Learning to Detect Manipulated Facial Images". In: *International Conference on Computer Vision (ICCV)*.
- Saeidnia, Hamid Reza et al. (2025). "Artificial intelligence in the battle against disinformation and misinformation: a systematic review of challenges and approaches". In: *Knowledge and Information Systems* 67.4, pp. 3139–3158.
- Saikia, Pallabi et al. (2022). "A hybrid CNN-LSTM model for video deepfake detection by leveraging optical flow features". In: *2022 international joint conference on neural networks (IJCNN)*. IEEE, pp. 1–7.
- Salehinejad, Hojjat et al. (2017). "Recent advances in recurrent neural networks". In: *arXiv preprint arXiv:1801.01078*.
- Sipser, Michael (2013). *Introduction to the Theory of Computation*. Third. Boston, MA: Course Technology. ISBN: 113318779X.
- Soudy, Ahmed Hatem et al. (2024). "Deepfake detection using convolutional vision transformers and convolutional neural networks". In: *Neural Computing and Applications* 36.31, pp. 19759–19775.
- Stephen, Daniel and Teddy Mantoro (2022). "Usage of convolutional neural network for deepfake video detection with face-swapping technique". In: *2022 5th International Conference of Computer and Informatics Engineering (IC2IE)*. IEEE, pp. 22–28.
- Taeb, Maryam and Hongmei Chi (2022). "Comparison of deepfake detection techniques through deep learning". In: *Journal of Cybersecurity and Privacy* 2.1, pp. 89–106.
- Tipper, Sarah, Hany F Atlam, and Harjinder Singh Lallie (2024). "An investigation into the utilisation of CNN with LSTM for video deepfake detection". In: *Applied Sciences* 14.21, p. 9754.

- Vaswani, Ashish et al. (2017). "Attention is all you need". In: *Advances in neural information processing systems* 30.
- Weerawardana, MC and TGI Fernando (2021). "Deepfakes detection methods: A literature survey". In: *2021 10th International Conference on Information and Automation for Sustainability (ICIAfS)*. IEEE, pp. 76–81.
- Yasser, Basma et al. (2023). "Deepfake Detection Using EfficientNet and XceptionNet". In: *2023 Eleventh International Conference on Intelligent Computing and Information Systems (ICICIS)*. IEEE, pp. 598–603.
- Zou, Jinming, Yi Han, and Sung-Sau So (2009). "Overview of artificial neural networks". In: *Artificial neural networks: methods and applications*, pp. 14–22.

1 Appendix A (Ethics form)

Do not amend before use

Middlesex University London

Research Ethics Screening Form for Students
Only for students on taught programmes – e.g., BSc, MSc, MA, LL.M etc
NOT for Postgraduate Researchers – e.g., MRes/MPH/PhD degrees

Middlesex University is concerned with protecting the rights, health, safety, dignity, and privacy of its research participants. It is also concerned with protecting the health, safety, rights, and academic freedom of its students and with safeguarding its own reputation for conducting high quality, ethical research.

This Research Ethics Screening Form will enable students to self-assess and determine whether the research requires ethical review and approval via the Middlesex Online Research Ethics (MORE) form before commencing the study. Supervisors must approve this form after consultation with students.

Student Name:	Chetan Bissessor	Email:	cb1497@live.mdx.ac.uk
Research project title:	A Complexity-Based Analysis of Deepfake Detection Mechanisms		
Programme of study/module:	MSc Cyber Security and Pen Testing		
Supervisor Name:	Dr Andrew Lewis-Smith	Email:	A.Lewis-Smith@mdx.ac.uk

Please answer whether your research/study involves any of the following given below:

1. "ANIMALS or animal parts.	☐ Yes	☒ No
2. "CELL LINES (established and commercially available cells - biological research).	☐ Yes	☒ No
3. "CELL CULTURE (Primary: from animal/human cells- biological research).	☐ Yes	☒ No
4. "CLINICAL Audits or Assessments (e.g. in medical settings).	☐ Yes	☒ No

5. "CONFLICT of INTEREST or lack of IMPARTIALITY. If unsure see "Code of Practice for Research" (Sec 3.5) at: https://unihub.mdx.ac.uk/study/spotlights/types/research-at-middlesex/research-ethics	☐ Yes	☒ No
6. "DATA to be used that is not freely available (e.g. secondary data needing permission for access or use).	☐ Yes	☒ No
7. "DAMAGE (e.g. to precious artefacts on to the environment) or present a significant risk to society).	☐ Yes	☒ No
8. "EXTERNAL ORGANISATION – research carried out within an external organisation or your research is commissioned by a government (or government body).	☐ Yes	☒ No
9. "FIELDWORK (e.g biological research, ethnography studies).	☐ Yes	☒ No
10. "GENETICALLY MODIFIED ORGANISMS (GMOs) (biological research).	☐ Yes	☒ No
11. "GENE THERAPY including DNA sequenced data (biological research).	☐ Yes	☒ No
12. "HUMAN PARTICIPANTS – ANONYMOUS questionnaires (participants not identified or identifiable).	☐ Yes	☒ No

University Ethics Committee (Dr Caroline George) - Ver 6, August 2024

1/2

Figure 1: Ethics form

Do not amend before use

Middlesex University London

13. "HUMAN PARTICIPANTS – IDENTIFIABLE (participants are identified or can be identified): survey/questionnaire/INTERVIEWS/ focus groups/ experiments/ observation studies/ evaluation studies.	☐ Yes	☒ No
14. "HUMAN TISSUE (e.g., human relevant material, e.g., blood, saliva, urine, breast milk, faecal material).	☐ Yes	☒ No
15. "ILLEGAL/HARMFUL activities research (e.g., development of technology intended to be used in an illegal/harmful context or to breach security systems, searching the internet for information on highly sensitive topics such as child and extreme pornography, terrorism, use of the DARK WEB, research harmful to national security).	☐ Yes	☒ No

Figure 2: Ethics form

16. PERMISSION is required to access premises or research participants.	☐ Yes	☐ No
17. PERSONAL DATA PROCESSING (Any activity with data that can directly or indirectly identify a living person). For example data gathered from interviews, databases, digital devices such as mobile phones, social media or internet platforms or apps with or without individuals' owners' knowledge or consent, and/or could lead to individuals/owners being IDENTIFIED or SPECIAL CATEGORY DATA (GDPR) or CRIMINAL OFFENCE DATA. *Special category data (GDPR- Art3): "personal data revealing racial or ethnic origin, political opinions, religious or philosophical beliefs, or trade union membership, and the processing of genetic data, biometric data for the purpose of uniquely identifying a natural person, data concerning health or data concerning a natural person's sex life or sexual orientation".	☐ Yes	☐ No
18. PUBLIC WORKS DOCTORATES: Evidence of permission is required for use of work/artifacts that are protected by Intellectual Property (IP) Rights (e.g. copyright, design right) in a doctoral critical commentary when the IP in the work/artifacts jointly prepared/produced or is owned by another body	☐ Yes	☐ No
19. RISK OF PHYSICAL OR PSYCHOLOGICAL HARM (e.g., TRAVEL to dangerous places in your own country or in a foreign country (see https://www.gov.uk/foreign-travel-advice), research with NGOs/humanitarian groups in conflict/dangerous zones, development of technology/agent/chemical that may be harmful to others, any other foreseeable dangerous risks).	☐ Yes	☐ No
20. SECURITY CLEARANCE – required for research.	☐ Yes	☐ No
21. SENSITIVE TOPICS (e.g., anything deeply personal and distressing, taboo, intrusive, stigmatising, sexual in nature, potentially dangerous, etc).	☐ Yes	☐ No

M – Minimal Risk; X – More than Minimal Risk; H – High Risk

If you have answered 'yes' to ANY of the items in the table, your application REQUIRES ethical review and approval using the MOREform BEFORE commencing your research. Please apply for ethical approval using the MOREform (<https://moreform.mdx.ac.uk/>). Consult your supervisor for guidance. Also see Middlesex Online Research Ethics (MyLearning area) and www.bmy.com/ethics

If you have answered 'No' to ALL of the items in the table, your application is Low Risk and you may NOT require ethical review and approval using the MOREform before commencing your research. Your research supervisor will confirm this below.

Student Signature:  Date: 04 April 2025

Figure 3: Ethics form

Do not amend before use



To be completed by the supervisor:

Based on the details provided in the self-assessment form, I confirm that:	Insert Y or N
The study is Low Risk and does not require ethical review & approval using the MOREform	Y
The study requires ethical review and approval using the MOREform	N

Supervisor Signature: ... Andrew Lewis-Smith Date: ... 04/04/2025

Figure 4: Ethics form

.2 Appendix B (Codes used)

Data processing code for Celeb-DF-v2:

Listing 1: `celebDFv2DataPreprocessing`

```

1 import os
2 import cv2
3 import numpy as np
4 import pandas as pd
5 from google.colab import drive
6
7 # Mount Google Drive
8 drive.mount('/content/drive')
9
10 # Defining paths
11 BASE_DIR = '/content/drive/MyDrive/Dissertation'
12
13 DATASETS_PATH = os.path.join(BASE_DIR, 'Datasets')
14 PREPROCESSING_PATH = os.path.join(BASE_DIR, 'Preprocessing')
15 MODELS_PATH = os.path.join(BASE_DIR, 'Models')
16 RESULTS_PATH = os.path.join(BASE_DIR, 'Results')
17
18 # Path to the specific dataset folder
19 CELEB_DF_V2_PATH = os.path.join(DATASETS_PATH, 'celeb_df_v2')
20
21 # Paths for subfolders within Preprocessing

```

```

22 FRAMES_PATH = os.path.join(PREPROCESSING_PATH, 'frames_extracted')
23 PREPROCESSED_DATA_PATH = os.path.join(PREPROCESSING_PATH, 'preprocessed_data')
24
25 # Ensure all relevant directories exist
26 os.makedirs(FRAMES_PATH, exist_ok=True)
27 os.makedirs(PREPROCESSED_DATA_PATH, exist_ok=True)
28
29 # Downloading a pre-trained face-detector
30 FACE_CASCADE_PATH = '/content/haarcascade_frontalface_default.xml'
31 if not os.path.exists(FACE_CASCADE_PATH):
32     !wget https://raw.githubusercontent.com/opencv/opencv/master/data/
33         haarcascades/haarcascade_frontalface_default.xml -O {FACE_CASCADE_PATH}
34     print("Downloaded face cascade classifier.")
35
36 print("Environment setup complete.")
37
38 # Path to the list of testing videos
39 list_file_path = os.path.join(CELEB_DF_V2_PATH, 'List_of_testing_videos.txt')
40
41 # Check if the list file exists
42 if not os.path.exists(list_file_path):
43     print(f"ERROR: The file '{list_file_path}' was not found. Please ensure your
44         dataset is correctly placed.")
45 else:
46     print("List of testing videos found. Proceeding with labeling.")
47
48 # Create a DataFrame to store video info
49 video_info = []
50
51 with open(list_file_path, 'r') as f:
52     for line in f:
53         line = line.strip()
54         if not line:
55             continue
56         parts = line.split()
57         label = int(parts[0]) # 0 for real, 1 for fake
58         video_path_in_list = parts[1]
59         video_name = os.path.basename(video_path_in_list)
60
61         # Map the video to its respective directory within the celeb_df_v2
62         # folder
63         # Note: The video path in the list file already contains the subfolder
64         # names.
65         # We just need to join it with the base path to the celeb_df_v2 folder.
66         full_video_path = os.path.join(CELEB_DF_V2_PATH, video_path_in_list)
67
68         video_info.append({
69             'video_name': video_name,
70             'video_path': full_video_path,

```

```

67         'label': label,
68         'label_name': 'fake' if label == 1 else 'real'
69     })
70
71 df_videos = pd.DataFrame(video_info)
72 print(f"Found {len(df_videos)} videos in the list.")
73 df_videos.head()
74
75 # Define a face cascade classifier
76 face_cascade = cv2.CascadeClassifier(FACE_CASCADE_PATH)
77
78 # Define the target size for the images (Xception's input)
79 TARGET_SIZE = (299, 299)
80
81 # Define the interval for frame extraction (e.g., every Nth frame)
82 FRAME_INTERVAL = 10
83
84 all_frames_data = []
85
86 print("Starting video processing...")
87
88 for index, row in df_videos.iterrows():
89     video_path = row['video_path']
90     video_label = row['label']
91     video_name = row['video_name']
92
93     # Open the video file
94     cap = cv2.VideoCapture(video_path)
95     if not cap.isOpened():
96         print(f"Warning: Could not open video file: {video_path}")
97         continue
98
99     frame_count = 0
100     frames_saved = 0
101
102     while True:
103         ret, frame = cap.read()
104         if not ret:
105             break # End of video
106
107         # Process a frame every FRAME_INTERVAL
108         if frame_count % FRAME_INTERVAL == 0:
109             # Convert the frame to grayscale for face detection
110             gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
111
112             # Detect faces in the grayscale frame
113             faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1,
114                                                  minNeighbors=5, minSize=(30, 30))

```

```

115         if len(faces) > 0:
116             # Take the first detected face (assuming one face per frame)
117             (x, y, w, h) = faces[0]
118
119             # Crop the face from the original color frame
120             cropped_face = frame[y:y+h, x:x+w]
121
122             # Resize the cropped face to the target size
123             resized_face = cv2.resize(cropped_face, TARGET_SIZE)
124
125             # Define a unique filename
126             frame_filename = f"{os.path.splitext(video_name)[0]}_frame_{
127                 frame_count}_{video_label}.jpg"
128             frame_path = os.path.join(PREPROCESSED_DATA_PATH, frame_filename
129                                     )
130
131             # Save the resized face image
132             cv2.imwrite(frame_path, resized_face)
133
134             # Store information about the saved frame
135             all_frames_data.append({
136                 'frame_filename': frame_filename,
137                 'original_video': video_name,
138                 'label': video_label
139             })
140
141             frames_saved += 1
142
143             frame_count += 1
144
145         cap.release()
146         print(f"Processed video '{video_name}'. Saved {frames_saved} frames.")
147
148     print("\nPreprocessing complete.")
149
150     import pandas as pd
151
152     # Create the final DataFrame from the collected data
153     df_preprocessed = pd.DataFrame(all_frames_data)
154
155     # Path to save the CSV file
156     csv_filename = 'celeb_df_v2_preprocessed_labels.csv'
157     csv_path = os.path.join(PREPROCESSING_PATH, csv_filename)
158
159     # Save the DataFrame to a CSV file
160     df_preprocessed.to_csv(csv_path, index=False)
161
162     print(f"Total processed frames: {len(df_preprocessed)}")
163     print(f"Saved dataset labels to: {csv_path}")

```

```

162
163 # Display the head of the final DataFrame
164 df_preprocessed.head()

```

Listing 2: FaceForensics++ Preprocessing

```

1  !pip install --upgrade pip
2  !pip install joblib==1.4.0 mtcnn lz4
3  !pip install tensorflow
4
5  import os
6  import cv2
7  from os.path import join
8  import pandas as pd
9  from tqdm.notebook import tqdm
10 import multiprocessing
11 import numpy as np
12 import tensorflow as tf
13 from mtcnn.mtcnn import MTCNN
14 from google.colab import drive
15 from joblib import Parallel, delayed
16 import logging
17 import threading
18 import time
19 import sys
20
21 # --- 1. SET THE CHUNK AND FRAME SKIP NUMBER ---
22 # Change this number to process a different chunk
23 CHUNK_NUMBER = 8
24 # Change this number to process fewer/more frames per video
25 # e.g., 5 means one out of every 5 frames is processed
26 FRAME_SKIP_INTERVAL = 8
27
28 # --- 2. Mount Google Drive ---
29 drive.mount('/content/drive')
30
31 # --- 3. Define Paths ---
32 CHUNKS_PATH = '/content/drive/MyDrive/Dissertation/Datasets/ff_chunks'
33 PREPROCESSED_DATA_PATH = '/content/drive/MyDrive/Dissertation/Datasets/
    FaceForensics++_preprocessed_v1'
34
35 # Silence TensorFlow/Keras logs
36 os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
37 logging.getLogger("tensorflow").setLevel(logging.ERROR)
38
39 # 4. Spinner Thread Function
40 stop_spinner = threading.Event()
41 def spinner():
42     """Simple spinner to indicate the program is still running."""

```

```

43     chars = ['|', '/', '-', '\\']
44     while not stop_spinner.is_set():
45         for char in chars:
46             sys.stdout.write(f'\rProcessing... {char}')
47             sys.stdout.flush()
48             time.sleep(0.1)
49             if stop_spinner.is_set():
50                 break
51     sys.stdout.write('\rDone. \n')
52     sys.stdout.flush()
53
54 # 5. Function to Extract Faces with MTCNN
55 def extract_faces_mtcnn_worker(video_path, output_dir, frame_skip_interval):
56     try:
57         detector_instance = MTCNN()
58
59         if not os.path.exists(video_path):
60             return "File not found"
61
62         cap = cv2.VideoCapture(video_path)
63         frame_count = 0
64         while True:
65             ret, frame = cap.read()
66             if not ret:
67                 break
68
69             # --- The code for skipping frames ---
70             if frame_count % frame_skip_interval != 0:
71                 frame_count += 1
72                 continue
73
74             try:
75                 frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
76                 detections = detector_instance.detect_faces(frame_rgb)
77             except Exception as e:
78                 print(f"Error processing frame {frame_count} from video {
79                     video_path}: {e}")
80                 frame_count += 1
81                 continue
82
83             if len(detections) == 1:
84                 detection = detections[0]
85                 x, y, width, height = detection['box']
86                 x1, y1 = abs(x), abs(y)
87                 x2, y2 = x1 + width, y1 + height
88                 x1 = max(0, x1)
89                 y1 = max(0, y1)
90                 x2 = min(frame_rgb.shape[1], x2)
91                 y2 = min(frame_rgb.shape[0], y2)

```

```

91         w = x2 - x1
92         h = y2 - y1
93
94         if w > 0 and h > 0:
95             cropped_face = frame[y1:y2, x1:x2]
96             output_filename = f"{os.path.basename(video_path).split('.')[0]}_{frame_count}.jpg"
97             output_filepath = join(output_dir, output_filename)
98             cv2.imwrite(output_filepath, cropped_face)
99
100         frame_count += 1
101         cap.release()
102         return "Success"
103     except Exception as e:
104         print(f"CRITICAL ERROR: Failed to process video {video_path}. Error: {e}")
105         return "Failed"
106
107 # 6. Main Processing Loop for a Single Chunk
108 if __name__ == '__main__':
109     num_workers = multiprocessing.cpu_count()
110     print(f"Using {num_workers} parallel workers.")
111     print(f"Processing every {FRAME_SKIP_INTERVAL}th frame.")
112
113     # --- Read the list of video paths for the specified chunk ---
114     chunk_dir = os.path.join(CHUNKS_PATH, f'ff_{CHUNK_NUMBER}')
115     chunk_file_path = os.path.join(chunk_dir, 'chunk_videos.txt')
116
117     if not os.path.exists(chunk_file_path):
118         print(f"Error: Chunk file not found at {chunk_file_path}")
119     else:
120         with open(chunk_file_path, 'r') as f:
121             video_paths_list = [line.strip() for line in f.readlines()]
122
123         print(f"Processing chunk {CHUNK_NUMBER} containing {len(video_paths_list)} videos.")
124
125         # --- Create output directories for the preprocessed frames ---
126         output_dir_real = os.path.join(PREPROCESSED_DATA_PATH, f'ff_{CHUNK_NUMBER}', 'real')
127         output_dir_fake = os.path.join(PREPROCESSED_DATA_PATH, f'ff_{CHUNK_NUMBER}', 'fake')
128         os.makedirs(output_dir_real, exist_ok=True)
129         os.makedirs(output_dir_fake, exist_ok=True)
130
131         tasks = []
132         for video_path in video_paths_list:
133             # Determine the label (real or fake) from the path to get the
134             # correct output folder

```



```

134         label = 'fake' if 'manipulated_sequences' in video_path else 'real'
135         output_dir = output_dir_real if label == 'real' else output_dir_fake
136
137         # Pass the frame_skip_interval to the worker
138         tasks.append(delayed(extract_faces_mtcnn_worker)(video_path,
139                                                             output_dir, FRAME_SKIP_INTERVAL))
140
141         # Start the spinner thread
142         stop_spinner.clear()
143         spinner_thread = threading.Thread(target=spinner)
144         spinner_thread.start()
145
146         try:
147             # Run the parallel process for the current chunk
148             processed_list = Parallel(
149                 n_jobs=num_workers,
150                 backend='loky'
151             )(tqdm(tasks, desc=f"Chunk {CHUNK_NUMBER}", total=len(tasks)))
152         finally:
153             # Signal the spinner to stop and wait for it to finish
154             stop_spinner.set()
155             spinner_thread.join()
156
157         print(f"\nPreprocessing for chunk {CHUNK_NUMBER} completed. Face frames
158               saved.")

```

Listing 3: Hybrid Model Preprocessing

```

1  import os
2  import pandas as pd
3  import numpy as np
4  import cv2
5  import dlib
6
7  # Mount Google Drive
8  from google.colab import drive
9  drive.mount('/content/drive')
10
11 # Define paths
12 BASE_DIR = '/content/drive/MyDrive/Dissertation'
13 CELEB_DF_DIR = os.path.join(BASE_DIR, 'Datasets', 'celeb_df_v2')
14 PREPROCESSING_PATH = os.path.join(BASE_DIR, 'Preprocessing')
15 RNN_PREPROCESSED_DATA_PATH = os.path.join(PREPROCESSING_PATH, '
    rnn_preprocessed_data')
16
17 # Create directories if they don't exist
18 os.makedirs(RNN_PREPROCESSED_DATA_PATH, exist_ok=True)
19
20 print("Environment setup and imports complete.")

```

```

21
22 # Load the list of video paths and their labels from the text file
23 video_list_path = os.path.join(CELEB_DF_DIR, 'List_of_testing_videos.txt')
24
25 # Read the text file and parse the data
26 video_data = []
27 with open(video_list_path, 'r') as file:
28     for line in file:
29         parts = line.strip().split()
30         label = int(parts[0].replace('0', '0').replace('1', '1')) # Convert '0'
31                             and '1' to integers 0 and 1
32         video_path = parts[1]
33         video_data.append({'video_path': video_path, 'label': label})
34
35 # Create a DataFrame from the parsed data
36 df_videos = pd.DataFrame(video_data)
37
38 print(f"Loaded {len(df_videos)} videos.")
39 print("DataFrame head:")
40 print(df_videos.head())
41
42 SEQUENCE_LENGTH = 20 # Number of frames to extract per video
43 MIN_VALID_FRAMES = 1 # Minimum number of valid frames required to save a
44                         sequence
45 FACE_SIZE = (299, 299) # Target face size for Xception input
46
47 # Initialize dlib's face detector
48 detector = dlib.get_frontal_face_detector()
49
50 processed_video_data = []
51
52 from tqdm import tqdm # this includes a progress bar in the output
53
54 # Iterate through each video in the DataFrame with a progress bar
55 for index, row in tqdm(df_videos.iterrows(), total=len(df_videos), desc="
56     Processing videos"):
57     video_local_path = os.path.join(CELEB_DF_DIR, row['video_path'])
58     video_name = os.path.basename(video_local_path)
59     video_label = row['label']
60
61     cap = cv2.VideoCapture(video_local_path)
62     if not cap.isOpened():
63         print(f"\nCould not open video: {video_local_path}. Skipping.")
64         continue
65
66     total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

```

```

67     if total_frames < SEQUENCE_LENGTH:
68         print(f"\nVideo {video_name} is too short ({total_frames} frames).
           Skipping.")
69         cap.release()
70         continue
71
72     frame_indices = np.linspace(0, total_frames - 1, SEQUENCE_LENGTH, dtype=int)
73
74     sequence_frames_path = os.path.join(RNN_PREPROCESSED_DATA_PATH, video_name)
75     os.makedirs(sequence_frames_path, exist_ok=True)
76
77     sequence_data = []
78
79     for i, frame_idx in enumerate(frame_indices):
80         cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
81         ret, frame = cap.read()
82         if not ret:
83             continue
84
85         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
86         faces = detector(gray, 1)
87
88         if len(faces) == 1:
89             face = faces[0]
90             x1, y1, x2, y2 = face.left(), face.top(), face.right(), face.bottom
               ()
91             cropped_face = frame[y1:y2, x1:x2]
92
93             if cropped_face.shape[0] == 0 or cropped_face.shape[1] == 0:
94                 continue
95
96             resized_face = cv2.resize(cropped_face, FACE_SIZE)
97
98             frame_filename = f"{os.path.splitext(video_name)[0]}_frame_{i}.jpg"
99             frame_save_path = os.path.join(sequence_frames_path, frame_filename)
100             cv2.imwrite(frame_save_path, resized_face)
101             sequence_data.append(frame_filename)
102         else:
103             continue
104
105     cap.release()
106
107     if len(sequence_data) >= MIN_VALID_FRAMES:
108         processed_video_data.append({
109             'video_name': video_name,
110             'label': video_label,
111             'sequence_frames': sequence_data,
112             'frames_path': sequence_frames_path
113         })

```

```

114         else:
115             print(f"\nSkipped video {video_name} due to an incomplete sequence ({len
                (sequence_data)} frames found).")
116
117 print("\nPreprocessing complete.")
118 print(f"Successfully processed {len(processed_video_data)} videos into sequences
        .")
119
120 # Create the final DataFrame from the collected data
121 df_rnn_preprocessed = pd.DataFrame(processed_video_data)
122
123 # Save the DataFrame to a CSV file
124 csv_filename = 'celeb_df_v2_rnn_preprocessed_labels.csv'
125 csv_path = os.path.join(PREPROCESSING_PATH, csv_filename)
126 df_rnn_preprocessed.to_csv(csv_path, index=False)
127
128 print(f"Dataset analysis for hybrid model saved to: {csv_path}")
129 print(df_rnn_preprocessed.head())
130
131 import pandas as pd
132 import os
133 import ast
134
135 # Define paths
136 PREPROCESSING_PATH = '/content/drive/MyDrive/Dissertation/Preprocessing'
137 csv_filename = 'celeb_df_v2_rnn_preprocessed_labels.csv'
138 csv_path = os.path.join(PREPROCESSING_PATH, csv_filename)
139
140 # Load the full dataset
141 df_rnn_preprocessed = pd.read_csv(csv_path)
142
143
144 FINAL_MIN_FRAMES = 8
145
146
147 #Convert sequence_frames to a list
148 df_rnn_preprocessed['sequence_frames'] = df_rnn_preprocessed['sequence_frames'].
    apply(ast.literal_eval)
149
150 # Filter the DataFrame
151 df_rnn_filtered = df_rnn_preprocessed[df_rnn_preprocessed['sequence_frames'].str
    .len() >= FINAL_MIN_FRAMES].copy()
152
153 # Print results
154 print(f"Original number of videos: {len(df_rnn_preprocessed)}")
155 print(f"Number of videos with at least {FINAL_MIN_FRAMES} frames: {len(
    df_rnn_filtered)}")
156 print("\nFiltered DataFrame head:")
157 print(df_rnn_filtered.head())

```

```

158
159 # Save the filtered dataset to a new CSV
160 filtered_csv_path = os.path.join(PREPROCESSING_PATH, f'
    celeb_df_v2_rnn_filtered_labels_min{FINAL_MIN_FRAMES}.csv')
161 df_rnn_filtered.to_csv(filtered_csv_path, index=False)
162
163 print(f"\nFiltered dataset saved to: {filtered_csv_path}")
164
165 import os
166
167 PREPROCESSING_PATH = '/content/drive/MyDrive/Dissertation/Preprocessing'
168 print("Contents of the Preprocessing folder:")
169 print(os.listdir(PREPROCESSING_PATH))

```

Listing 4: Data Cleaning and Downscaling

```

1 from google.colab import drive
2 drive.mount('/content/drive')
3
4 import os
5 from PIL import Image
6 from concurrent.futures import ThreadPoolExecutor, as_completed
7 from tqdm.auto import tqdm
8
9 # Define the root directory where your images are stored.
10 root_dir = '/content/drive/MyDrive/Dissertation/ff_unified'
11
12 def check_and_delete(file_path):
13     """Function to be executed by a worker thread."""
14     if not file_path.lower().endswith(('.png', '.jpg', '.jpeg', '.gif', '.bmp')):
15         :
16         return f"Skipped non-image file: {file_path}"
17
18     try:
19         with Image.open(file_path) as img:
20             img.verify() # Check for integrity
21             return f"File is valid: {file_path}"
22     except (IOError, OSError, Image.UnidentifiedImageError) as e:
23         print(f"\nCorrupted file detected and deleted: {file_path}")
24         os.remove(file_path)
25         return f"Corrupted: {file_path}"
26
27 # Check if the directory exists
28 if not os.path.exists(root_dir):
29     print(f"Directory not found: {root_dir}")
30 else:
31     # Gather all file paths first
32     file_paths = []
33     for dirpath, _, filenames in os.walk(root_dir):

```

```

33     for filename in filenames:
34         file_paths.append(os.path.join(dirpath, filename))
35
36 if not file_paths:
37     print("No files found to process.")
38 else:
39     # Determine the number of workers. A good heuristic for I/O bound tasks
40     # is a few times the number of CPU cores.
41     num_workers = os.cpu_count() * 2 if os.cpu_count() else 4
42     print(f"Using {num_workers} threads to check {len(file_paths)} files..."
43           )
44
45     with ThreadPoolExecutor(max_workers=num_workers) as executor:
46         futures = {executor.submit(check_and_delete, file_path) for
47                     file_path in file_paths}
48         for future in tqdm(as_completed(futures), total=len(file_paths),
49                           desc="Checking files"):
50             pass # The progress bar is managed by tqdm
51
52 print("\nCorrupted file check complete.")
53
54 import os
55 import shutil
56 import time
57 from tqdm import tqdm
58
59 def organize_files_into_subfolders(source_dir, batch_size=1000, prefix="batch_")
60 :
61     """
62     Organizes files in a directory into subfolders.
63
64     Args:
65         source_dir (str): The path to the directory containing the files.
66         batch_size (int): The number of files to put in each subfolder.
67         prefix (str): The prefix for the new subfolder names.
68     """
69     print(f"Starting organization of files in: {source_dir}")
70     if not os.path.exists(source_dir):
71         print(f"Error: Source directory '{source_dir}' does not exist.")
72         return
73
74     # Get a list of all files in the source directory
75     all_files = [f for f in os.listdir(source_dir) if os.path.isfile(os.path.
76         join(source_dir, f))]
77     all_files.sort() # Sort to ensure a consistent order
78
79     num_files = len(all_files)
80     print(f"Found {num_files} files to organize.")

```

```

77     # Process files in batches
78     for i in tqdm(range(0, num_files, batch_size), desc="Organizing files into
subfolders"):
79         # Define the current batch of files
80         batch_files = all_files[i:i + batch_size]
81
82         # Create a new subfolder for this batch
83         folder_name = f"{prefix}{i // batch_size:04d}"
84         destination_dir = os.path.join(source_dir, folder_name)
85
86         # Check if folder already exists
87         if not os.path.exists(destination_dir):
88             os.makedirs(destination_dir)
89
90         # Move each file in the batch
91         for filename in batch_files:
92             source_path = os.path.join(source_dir, filename)
93             destination_path = os.path.join(destination_dir, filename)
94
95             try:
96                 shutil.move(source_path, destination_path)
97             except Exception as e:
98                 print(f"Error moving {filename}: {e}")
99
100         # Optional: Add a small delay to avoid overwhelming the Google Drive
connection
101         # time.sleep(0.5)
102
103     print("\nFile organization complete.")
104
105
106 base_path = '/content/drive/MyDrive/Dissertation/ff_unified'
107 fake_path = os.path.join(base_path, 'fake')
108 real_path = os.path.join(base_path, 'real')
109
110 # Organize the 'fake' folder
111 organize_files_into_subfolders(fake_path, batch_size=15000, prefix="fake_batch_"
)
112
113 # Organize the 'real' folder
114 organize_files_into_subfolders(real_path, batch_size=15000, prefix="real_batch_"
)
115
116 print("\nAll files have been organized. You can now run your main training
script.")
117
118 import os
119 from PIL import Image
120 from concurrent.futures import ProcessPoolExecutor

```

```

121 from tqdm.auto import tqdm
122 import multiprocessing
123
124 # Define the root directory where your images are stored.
125 root_dir = '/content/drive/MyDrive/Dissertation/ff_unified'
126 new_size = (224, 224)
127
128 def resize_image(file_path):
129     """Function to be executed by a worker process."""
130     if not file_path.lower().endswith(('.png', '.jpg', '.jpeg', '.gif', '.bmp')):
131         :
132         return None
133
134     try:
135         with Image.open(file_path) as img:
136             img = img.resize(new_size, Image.Resampling.LANCZOS)
137             img.save(file_path)
138             return file_path
139     except Exception:
140         return None
141
142 # Check if the directory exists
143 if not os.path.exists(root_dir):
144     print(f"Directory not found: {root_dir}")
145 else:
146     # Gather all file paths first
147     file_paths = []
148     for dirpath, _, filenames in os.walk(root_dir):
149         for filename in filenames:
150             file_paths.append(os.path.join(dirpath, filename))
151
152     if not file_paths:
153         print("No files found to process.")
154     else:
155         # The number of workers is typically the number of CPU cores for CPU-
156         # bound tasks
157         num_workers = multiprocessing.cpu_count()
158         print(f"Using {num_workers} processes to resize {len(file_paths)} images
159             ...")
160
161         with ProcessPoolExecutor(max_workers=num_workers) as executor:
162             list(tqdm(executor.map(resize_image, file_paths), total=len(
163                 file_paths), desc="Resizing images"))
164
165 print("\nImage resizing complete.")

```

Listing 5: CNN Deepfake Detection for Celeb-DF-v2


```

2 import os
3 import pandas as pd
4 import numpy as np
5 import time
6 import psutil
7 import json
8 import matplotlib.pyplot as plt
9
10 # TensorFlow and Keras imports
11 import tensorflow as tf
12 from tensorflow.keras.models import Model
13 from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
14 from tensorflow.keras.applications import Xception
15 from tensorflow.keras.preprocessing.image import ImageDataGenerator
16 from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
17
18 # Scikit-learn imports for data splitting and evaluation
19 from sklearn.model_selection import train_test_split
20 from sklearn.metrics import accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix
21
22 # Mount Google Drive
23 from google.colab import drive
24 drive.mount('/content/drive')
25
26 #Helper function to get memory usage
27 def get_memory_usage():
28     """Returns memory usage in MB."""
29     process = psutil.Process(os.getpid())
30     return process.memory_info().rss / 1024 / 1024
31
32 # Defining paths
33 BASE_DIR = '/content/drive/MyDrive/Dissertation'
34 PREPROCESSING_PATH = os.path.join(BASE_DIR, 'Preprocessing')
35 PREPROCESSED_DATA_PATH = os.path.join(PREPROCESSING_PATH, 'preprocessed_data')
36 MODELS_PATH = os.path.join(BASE_DIR, 'Models')
37 SAVED_MODELS_PATH = os.path.join(MODELS_PATH, 'saved_models')
38 RESULTS_PATH = os.path.join(BASE_DIR, 'Results')
39
40 # Create directories if they don't exist
41 os.makedirs(SAVED_MODELS_PATH, exist_ok=True)
42 os.makedirs(RESULTS_PATH, exist_ok=True)
43
44 print("Environment setup and imports complete.")
45
46 # Load the preprocessed labels CSV file
47 csv_file_path = os.path.join(PREPROCESSING_PATH, '
    celeb_df_v2_preprocessed_labels.csv')
48 df_data = pd.read_csv(csv_file_path)

```

```

49
50
51 df_data['frame_path'] = df_data['frame_filename'].apply(
52     lambda x: os.path.join(PREPROCESSED_DATA_PATH, x)
53 )
54
55
56 df_data['label'] = df_data['label'].astype(str)    # To Convert the label to a
57     string
58
59 # Perform the initial split into a training set and a temporary testing/
60     validation set
61 train_df, temp_df = train_test_split(df_data, test_size=0.3, random_state=42,
62     stratify=df_data['label'])
63
64 # Split the temporary set equally into a validation set and a final testing set
65 val_df, test_df = train_test_split(temp_df, test_size=0.5, random_state=42,
66     stratify=temp_df['label'])
67
68
69 print(f"\nTotal frames: {len(df_data)}")
70 print(f"Training set size: {len(train_df)}")
71 print(f"Validation set size: {len(val_df)}")
72 print(f"Testing set size: {len(test_df)}")
73
74 # Define batch size
75 BATCH_SIZE = 32
76
77
78 # Data augmentation for the training set
79 train_datagen = ImageDataGenerator(
80     rescale=1./255, # Scale pixel values to [0, 1]
81     rotation_range=20,
82     width_shift_range=0.2,
83     height_shift_range=0.2,
84     shear_range=0.2,
85     zoom_range=0.2,
86     horizontal_flip=True,
87     fill_mode='nearest'
88 )
89
90 # only rescaling for validation and test sets
91 val_test_datagen = ImageDataGenerator(rescale=1./255)
92
93 # Create generators with multiprocessing enabled
94 print("\nCreating data generators with multiple workers...")
95 train_generator = train_datagen.flow_from_dataframe(
96     dataframe=train_df,
97     x_col='frame_path',
98     y_col='label',
99     target_size=(299, 299), # Xception's input size

```

```

94     batch_size=BATCH_SIZE,
95     class_mode='binary',
96     use_multiprocessing=True, # Enable multiprocessing
97     workers=8 # Number of worker processes to use
98 )
99
100 validation_generator = val_test_datagen.flow_from_dataframe(
101     dataframe=val_df,
102     x_col='frame_path',
103     y_col='label',
104     target_size=(299, 299),
105     batch_size=BATCH_SIZE,
106     class_mode='binary',
107     use_multiprocessing=True,
108     workers=8
109 )
110
111 test_generator = val_test_datagen.flow_from_dataframe(
112     dataframe=test_df,
113     x_col='frame_path',
114     y_col='label',
115     target_size=(299, 299),
116     batch_size=BATCH_SIZE,
117     class_mode='binary',
118     shuffle=False, # Do not shuffle the test data
119     use_multiprocessing=True,
120     workers=8
121 )
122
123 print("Data generators created.")
124
125 # Load the pre-trained Xception model without its top layers
126 base_model = Xception(weights='imagenet', include_top=False, input_shape=(299,
127     299, 3))
128
129 # Freeze the layers of the base model
130 base_model.trainable = False
131
132 # Adding new classification layers
133 x = base_model.output
134 x = GlobalAveragePooling2D()(x)
135 x = Dense(128, activation='relu')(x)
136 x = Dropout(0.5)(x)
137 predictions = Dense(1, activation='sigmoid')(x) # Sigmoid for binary
138     classification
139
140 # Combine the base model and new layers
141 model = Model(inputs=base_model.input, outputs=predictions)

```

```

141 # Compile the model
142 model.compile(
143     optimizer='adam',
144     loss='binary_crossentropy',
145     metrics=['accuracy']
146 )
147
148 model.summary()
149
150 # Define callbacks
151 model_checkpoint_path = os.path.join(SAVED_MODELS_PATH, '
152     xception_best_model_multithreaded.h5')
153 checkpoint = ModelCheckpoint(
154     model_checkpoint_path,
155     monitor='val_accuracy',
156     save_best_only=True,
157     mode='max',
158     verbose=1
159 )
160
161 # --- Updated Early Stopping patience to 3 ---
162 early_stopping = EarlyStopping(
163     monitor='val_loss',
164     patience=3,
165     verbose=1
166 )
167
168 print("Starting model training...")
169 # --- Get memory usage before training ---
170 mem_before_training = get_memory_usage()
171 print(f"Memory usage before training: {mem_before_training:.2f} MB")
172
173 # Start timer
174 start_time = time.time()
175
176 # Train the model
177 history = model.fit(
178     train_generator,
179     epochs=10,
180     validation_data=validation_generator,
181     callbacks=[checkpoint, early_stopping]
182 )
183
184 # End timer and get memory usage after training
185 training_time = time.time() - start_time
186 mem_after_training = get_memory_usage()
187 mem_delta = mem_after_training - mem_before_training
188 print(f"Memory usage after training: {mem_after_training:.2f} MB")
189 print(f"Memory delta: {mem_delta:.2f} MB")

```

```

189 print(f"\nTraining completed in {training_time:.2f} seconds.")
190
191
192 history_path = os.path.join(RESULTS_PATH, '
      xception_training_history_multithreaded.json')
193 with open(history_path, 'w') as f:
194     json.dump(history.history, f)
195 print(f"Training history saved to: {history_path}")
196
197 # Load the best saved model
198 best_model = tf.keras.models.load_model(model_checkpoint_path)
199
200 print("Evaluating model on the test set...")
201
202 # Start timer for inference
203 inference_start_time = time.time()
204
205 # Get predictions on the test set
206 test_pred_probs = best_model.predict(test_generator)
207 test_preds = (test_pred_probs > 0.5).astype(int)
208
209 # End timer for inference
210 inference_time = time.time() - inference_start_time
211
212 # Get true labels from the test generator
213 true_labels = test_generator.classes
214
215 # Calculate metrics
216 acc = accuracy_score(true_labels, test_preds)
217 prec = precision_score(true_labels, test_preds)
218 rec = recall_score(true_labels, test_preds)
219 f1 = f1_score(true_labels, test_preds)
220 conf_matrix = confusion_matrix(true_labels, test_preds)
221
222 # Extract TP, FP, TN, FN from the confusion matrix
223 tn, fp, fn, tp = conf_matrix.ravel()
224
225 # Create a dictionary of results
226 results = {
227     'model': 'Xception_multithreaded',
228     'accuracy': float(acc),
229     'precision': float(prec),
230     'recall': float(rec),
231     'f1_score': float(f1),
232     'training_time_sec': float(training_time),
233     'inference_time_sec': float(inference_time),
234     'memory_before_training_mb': float(mem_before_training),
235     'memory_after_training_mb': float(mem_after_training),
236     'memory_delta_mb': float(mem_delta),

```

```

237     'true_positives': int(tp),
238     'false_positives': int(fp),
239     'true_negatives': int(tn),
240     'false_negatives': int(fn),
241     'confusion_matrix': conf_matrix.tolist()
242 }
243
244 print("\n--- Model Performance ---")
245 print(f"Accuracy: {acc:.4f}")
246 print(f"Precision: {prec:.4f}")
247 print(f"Recall: {rec:.4f}")
248 print(f"F1-Score: {f1:.4f}")
249 print("\n--- Confusion Matrix Components ---")
250 print(f"True Positives (TP): {tp}")
251 print(f"False Positives (FP): {fp}")
252 print(f"True Negatives (TN): {tn}")
253 print(f"False Negatives (FN): {fn}")
254 print("\nConfusion Matrix:")
255 print(conf_matrix)
256
257 # Save results to a file
258 results_path = os.path.join(RESULTS_PATH, 'xception_results_multithreaded.json')
259 with open(results_path, 'w') as f:
260     json.dump(results, f, indent=4)
261 print(f"\nResults saved to: {results_path}")
262
263 # Plotting the training history
264 if history.history:
265     # Create a figure with two subplots
266     plt.style.use('seaborn-v0_8-whitegrid')
267     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
268     fig.suptitle('Model Performance Over Epochs', fontsize=16)
269
270     # Plot training & validation accuracy values
271     ax1.plot(history.history['accuracy'])
272     ax1.plot(history.history['val_accuracy'])
273     ax1.set_title('Model Accuracy')
274     ax1.set_ylabel('Accuracy')
275     ax1.set_xlabel('Epoch')
276     ax1.legend(['Train', 'Validation'], loc='upper left')
277
278     # Plot training & validation loss values
279     ax2.plot(history.history['loss'])
280     ax2.plot(history.history['val_loss'])
281     ax2.set_title('Model Loss')
282     ax2.set_ylabel('Loss')
283     ax2.set_xlabel('Epoch')
284     ax2.legend(['Train', 'Validation'], loc='upper left')
285

```

```

286     # Use tight_layout() to prevent labels from overlapping
287     plt.tight_layout(rect=[0, 0.03, 1, 0.95])
288
289     # Save the plot
290     plot_path = os.path.join(RESULTS_PATH, '
        exception_training_history_multithreaded.png')
291     plt.savefig(plot_path)
292     print(f"Plot saved to: {plot_path}")
293 else:
294     print("Plotting skipped due to missing history data.")

```

Listing 6: Hybrid Model Deepfake Detection for Celeb-DF-v2

```

1
2 import os
3 import pandas as pd
4 import numpy as np
5 import tensorflow as tf
6 from tensorflow.keras.models import Model
7 from tensorflow.keras.layers import Dense, Dropout, LSTM, TimeDistributed, Input
8 from tensorflow.keras.optimizers import Adam
9 from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping
10 from tensorflow.keras.preprocessing.image import load_img, img_to_array
11 from sklearn.model_selection import train_test_split
12 from sklearn.metrics import accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix
13 import json
14 import time
15 import ast
16 import psutil
17 import matplotlib.pyplot as plt
18
19 # Mount Google Drive
20 from google.colab import drive
21 drive.mount('/content/drive')
22
23 # Define paths
24 BASE_DIR = '/content/drive/MyDrive/Dissertation'
25 PREPROCESSING_PATH = os.path.join(BASE_DIR, 'Preprocessing')
26 RNN_PREPROCESSED_DATA_PATH = os.path.join(PREPROCESSING_PATH, '
    rnn_preprocessed_data')
27 MODELS_PATH = os.path.join(BASE_DIR, 'Models')
28 RESULTS_PATH = os.path.join(BASE_DIR, 'Results')
29
30 # Ensure directories exist
31 os.makedirs(MODELS_PATH, exist_ok=True)
32 os.makedirs(RESULTS_PATH, exist_ok=True)
33
34 print("Setup complete.")

```

```

35
36 # Load the filtered dataset
37 csv_filename = 'celeb_df_v2_rnn_filtered_labels_min8.csv'
38 csv_path = os.path.join(PREPROCESSING_PATH, csv_filename)
39 df = pd.read_csv(csv_path)
40
41
42 # convert the sequence_frames column to a list from string
43 df['sequence_frames'] = df['sequence_frames'].apply(ast.literal_eval)
44
45 print(f"Loaded dataset with {len(df)} videos.")
46
47 # Split the data into training, validation, and test sets
48 train_df, temp_df = train_test_split(df, test_size=0.2, random_state=42,
49                                     stratify=df['label'])
49 val_df, test_df = train_test_split(temp_df, test_size=0.5, random_state=42,
50                                   stratify=temp_df['label'])
51
52 print(f"Training videos: {len(train_df)}")
53 print(f"Validation videos: {len(val_df)}")
54 print(f"Test videos: {len(test_df)}")
55
56 # --- Custom Data Generator for Sequences ---
57 from tensorflow.keras.utils import Sequence
58
59 class SequenceDataGenerator(Sequence):
60     def __init__(self, df, data_path, batch_size, sequence_length, face_size):
61         self.df = df
62         self.data_path = data_path
63         self.batch_size = batch_size
64         self.sequence_length = sequence_length
65         self.face_size = face_size
66         self.indices = np.arange(len(self.df))
67
68     def __len__(self):
69         return int(np.floor(len(self.df) / self.batch_size))
70
71     def __getitem__(self, index):
72         batch_indices = self.indices[index * self.batch_size:(index + 1) * self.
73             batch_size]
74         batch_df = self.df.iloc[batch_indices]
75
76         x_batch = []
77         y_batch = []
78
79         for _, row in batch_df.iterrows():
80             video_frames_path = os.path.join(self.data_path, row['video_name'])
81             sequence_frames_list = row['sequence_frames']

```



```

81         # Load images for the sequence
82         sequence_images = []
83         for frame_filename in sequence_frames_list:
84             img_path = os.path.join(video_frames_path, frame_filename)
85             if os.path.exists(img_path):
86                 img = load_img(img_path, target_size=self.face_size)
87                 img_array = img_to_array(img) / 255.0 # Normalize
88                 sequence_images.append(img_array)
89
90         # Pad or truncate the sequence to the fixed length
91         while len(sequence_images) < self.sequence_length:
92             sequence_images.append(np.zeros(self.face_size + (3,)))
93
94         if len(sequence_images) > self.sequence_length:
95             sequence_images = sequence_images[:self.sequence_length]
96
97         x_batch.append(np.stack(sequence_images, axis=0))
98         y_batch.append(row['label'])
99
100         return np.stack(x_batch, axis=0), np.array(y_batch)
101
102     # Define the data generators
103     BATCH_SIZE = 16
104     SEQUENCE_LENGTH = 20
105     FACE_SIZE = (299, 299)
106
107     train_generator = SequenceDataGenerator(train_df, RNN_PREPROCESSED_DATA_PATH,
108                                             BATCH_SIZE, SEQUENCE_LENGTH, FACE_SIZE)
109     val_generator = SequenceDataGenerator(val_df, RNN_PREPROCESSED_DATA_PATH,
110                                           BATCH_SIZE, SEQUENCE_LENGTH, FACE_SIZE)
111     test_generator = SequenceDataGenerator(test_df, RNN_PREPROCESSED_DATA_PATH,
112                                           BATCH_SIZE, SEQUENCE_LENGTH, FACE_SIZE)
113
114     print("\nData generators created.")
115
116     # Load the pre-trained Xception model without its top classification layers
117     xception_base = tf.keras.applications.Xception(weights='imagenet', include_top=
118             False, input_shape=FACE_SIZE + (3,))
119     xception_base.trainable = False # Freeze the layers
120
121     # Build the hybrid model
122     model_input = Input(shape=(SEQUENCE_LENGTH,) + FACE_SIZE + (3,))
123
124     # Apply the CNN (Xception) to each frame in the sequence
125     cnn_part = TimeDistributed(xception_base)(model_input)
126     cnn_part = TimeDistributed(tf.keras.layers.GlobalAveragePooling2D())(cnn_part)
127
128     # Process the sequence of features with an LSTM layer
129     lstm_part = LSTM(128, return_sequences=False)(cnn_part)

```

```

126 lstm_part = Dropout(0.5)(lstm_part)
127
128 # Final classification layers
129 output = Dense(1, activation='sigmoid')(lstm_part)
130
131 hybrid_model = Model(inputs=model_input, outputs=output)
132
133 hybrid_model.compile(optimizer=Adam(learning_rate=0.0001),
134                     loss='binary_crossentropy',
135                     metrics=['accuracy'])
136
137 hybrid_model.summary()
138
139 # Define callbacks
140 checkpoint_path = os.path.join(MODELS_PATH, 'hybrid_xception_lstm_model.h5')
141 checkpoint = ModelCheckpoint(checkpoint_path, monitor='val_accuracy', verbose=1,
142                             save_best_only=True, mode='max')
143 early_stopping = EarlyStopping(monitor='val_loss', patience=3, verbose=1, mode='
    min', restore_best_weights=True)
144
145 # Start timer and get initial memory usage
146 training_start_time = time.time()
147 process = psutil.Process(os.getpid())
148 memory_before_training_gb = process.memory_info().rss / (1024 ** 3)
149
150 print(f"Initial Memory Usage: {memory_before_training_gb:.2f} GB")
151
152 history = hybrid_model.fit(
153     train_generator,
154     epochs=10,
155     validation_data=val_generator,
156     callbacks=[checkpoint, early_stopping],
157 )
158
159 # End timer and get final memory usage
160 training_time = time.time() - training_start_time
161 memory_after_training_gb = process.memory_info().rss / (1024 ** 3)
162 memory_used_gb = memory_after_training_gb - memory_before_training_gb
163
164 print(f"Training finished. Total time: {training_time:.2f} seconds.")
165 print(f"Final Memory Usage: {memory_after_training_gb:.2f} GB")
166 print(f"Memory Used During Training: {memory_used_gb:.2f} GB")
167
168 print("\nEvaluating model on the test set:")
169
170 # Load the best saved model
171 best_model = tf.keras.models.load_model(checkpoint_path)

```

```

173
174 # Start timer for inference
175 inference_start_time = time.time()
176
177 # Get predictions on the test set
178 test_pred_probs = best_model.predict(test_generator)
179 test_preds = (test_pred_probs > 0.5).astype(int)
180
181 # End timer for inference
182 inference_time = time.time() - inference_start_time
183
184 # Get true labels from custom generator
185 true_labels = []
186 for i in range(len(test_generator)):
187     _, labels = test_generator.__getitem__(i)
188     true_labels.extend(labels)
189 true_labels = np.array(true_labels)
190
191 # Ensure sizes match before calculating metrics
192 if len(true_labels) != len(test_preds):
193     print(f"Warning: Mismatch in true labels ({len(true_labels)}) and
194           predictions ({len(test_preds)}).")
195     test_preds = test_preds[:len(true_labels)]
196
197 # Calculate metrics
198 acc = accuracy_score(true_labels, test_preds)
199 prec = precision_score(true_labels, test_preds)
200 rec = recall_score(true_labels, test_preds)
201 f1 = f1_score(true_labels, test_preds)
202 conf_matrix = confusion_matrix(true_labels, test_preds)
203
204 # Extract TP, FP, TN, FN from the confusion matrix
205 tn, fp, fn, tp = conf_matrix.ravel()
206
207 # Create a dictionary of results
208 results = {
209     'model': 'Hybrid_Xception_LSTM',
210     'accuracy': float(acc),
211     'precision': float(prec),
212     'recall': float(rec),
213     'f1_score': float(f1),
214     'training_time_sec': float(training_time),
215     'inference_time_sec': float(inference_time),
216     'memory_used_gb': float(memory_used_gb),
217     'true_positives': int(tp),
218     'false_positives': int(fp),
219     'true_negatives': int(tn),
220     'false_negatives': int(fn),
221     'confusion_matrix': conf_matrix.tolist()

```

```

221 }
222
223 # Save results to a file
224 results_path = os.path.join(RESULTS_PATH, 'hybrid_xception_lstm_results.json')
225 with open(results_path, 'w') as f:
226     json.dump(results, f, indent=4)
227 print(f"\nResults saved to: {results_path}")
228
229 # Plotting the training history
230 plt.style.use('seaborn-v0_8-whitegrid')
231 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
232 fig.suptitle('Hybrid Model Performance Over Epochs', fontsize=16)
233
234 # Plot training & validation accuracy values
235 ax1.plot(history.history['accuracy'])
236 ax1.plot(history.history['val_accuracy'])
237 ax1.set_title('Model Accuracy')
238 ax1.set_ylabel('Accuracy')
239 ax1.set_xlabel('Epoch')
240 ax1.legend(['Train', 'Validation'], loc='upper left')
241
242 # Plot training & validation loss values
243 ax2.plot(history.history['loss'])
244 ax2.plot(history.history['val_loss'])
245 ax2.set_title('Model Loss')
246 ax2.set_ylabel('Loss')
247 ax2.set_xlabel('Epoch')
248 ax2.legend(['Train', 'Validation'], loc='upper left')
249
250 plt.tight_layout(rect=[0, 0.03, 1, 0.95])
251
252 # Save the plot
253 plot_path = os.path.join(RESULTS_PATH, 'hybrid_xception_lstm_training_history.
    png')
254 plt.savefig(plot_path)
255
256 print(f"Plot saved to: {plot_path}")

```

Listing 7: CNN Detection FaceForensics++

```

1 from google.colab import drive
2 drive.mount('/content/drive')
3
4 !pip install tensorflow keras scikit-learn matplotlib tqdm pillow
5
6 import os
7 import pandas as pd
8 import numpy as np
9 import time

```

```

10 import json
11 import matplotlib.pyplot as plt
12 from tqdm import tqdm
13 import psutil
14 from concurrent.futures import ProcessPoolExecutor, as_completed
15
16 # TensorFlow and Keras imports
17 import tensorflow as tf
18 from tensorflow.keras.models import Model
19 from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
20 from tensorflow.keras.applications import Xception
21 from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
22
23 # Scikit-learn imports for data splitting and evaluation
24 from sklearn.model_selection import train_test_split
25 from sklearn.metrics import accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix
26
27 # Mount Google Drive
28 from google.colab import drive
29 drive.mount('/content/drive')
30
31 # Defining paths
32 BASE_DIR = '/content/drive/MyDrive/Dissertation'
33 DATASET_FOLDER = 'ff_unified'
34 DATASET_PATH = os.path.join(BASE_DIR, DATASET_FOLDER)
35 MODELS_PATH = os.path.join(BASE_DIR, 'Models')
36 RESULTS_PATH = os.path.join(BASE_DIR, 'Results')
37
38 os.makedirs(MODELS_PATH, exist_ok=True)
39 os.makedirs(RESULTS_PATH, exist_ok=True)
40
41 # Parameters
42 TARGET_SIZE = (299, 299)
43 BATCH_SIZE = 32
44 EPOCHS = 10
45 AUTOTUNE = tf.data.AUTOTUNE
46 NUM_WORKERS = os.cpu_count() // 2 or 1
47
48 print("Environment setup and imports complete.")
49 print("-" * 30)
50
51 # Getting file paths
52
53 def find_image_files(base_dir):
54     """Finds all image files in a directory and its subdirectories."""
55     image_files = []
56     for root, dirs, files in os.walk(base_dir):
57         for f in files:

```

```

58         if f.lower().endswith(('.png', '.jpg', '.jpeg')):
59             image_files.append(os.path.join(root, f))
60     return image_files
61
62 def collect_all_image_paths_multiprocessing(base_dir):
63     all_files = []
64     real_base_dir = os.path.join(base_dir, 'real')
65     fake_base_dir = os.path.join(base_dir, 'fake')
66
67     print("Collecting 'real' file paths from all subdirectories...")
68     real_files = find_image_files(real_base_dir)
69     print(f"Found {len(real_files)} real files.")
70
71     print("\nCollecting 'fake' file paths from all subdirectories...")
72     fake_files = find_image_files(fake_base_dir)
73     print(f"Found {len(fake_files)} fake files.")
74
75     return real_files, fake_files
76
77 print("Starting data collection...")
78 try:
79     real_files, fake_files = collect_all_image_paths_multiprocessing(
80         DATASET_PATH)
81 except OSError as e:
82     print(f"Error: {e}. An I/O error occurred while trying to list files.")
83     exit()
84
85 df_data = pd.DataFrame({
86     'frame_path': real_files + fake_files,
87     'label': [1] * len(real_files) + [0] * len(fake_files)
88 })
89
90 print(f"Total frames found: {len(df_data)}")
91
92 #Split dataset
93 train_df, temp_df = train_test_split(df_data, test_size=0.3, random_state=42,
94     stratify=df_data['label'])
95 val_df, test_df = train_test_split(temp_df, test_size=0.5, random_state=42,
96     stratify=temp_df['label'])
97
98 print(f"Training set size: {len(train_df)}")
99 print(f"Validation set size: {len(val_df)}")
100 print(f"Testing set size: {len(test_df)}")
101 print("-" * 30)
102
103 # TF Dataset - Loads a single image and handles corrupted files.
104 def load_image(path, label):
105     try:

```

```

104     img = tf.io.read_file(path)
105     img = tf.image.decode_image(img, channels=3, expand_animations=False)
106     img = tf.image.resize(img, TARGET_SIZE)
107     img = tf.cast(img, tf.float32) / 255.0
108     label_tensor = tf.cast(label, tf.int32)
109     # Return a tensor and a label
110     return img, label_tensor
111 except tf.errors.InvalidArgumentError:
112     # On a corrupted file, return a dummy tensor and a special label
113     # The filter function will remove these entries
114     print(f"Skipping empty or invalid image file: {path.numpy().decode('utf
115           -8')}}")
116     return tf.zeros(TARGET_SIZE + (3,), dtype=tf.float32), tf.constant(-1,
117                               dtype=tf.int32)
118
119 def create_tf_dataset(df, shuffle=False):
120     dataset = tf.data.Dataset.from_tensor_slices((df['frame_path'].tolist(), df[
121           'label'].tolist()))
122
123     if shuffle:
124         dataset = dataset.shuffle(buffer_size=len(df), reshuffle_each_iteration=
125               True)
126
127     def map_and_set_shape(x, y):
128         # tf.py_function to wrap the Python `load_image` function
129         image, label = tf.py_function(
130             load_image,
131             inp=[x, y],
132             Tout=[tf.float32, tf.int32]
133         )
134         image.set_shape(TARGET_SIZE + (3,))
135         label.set_shape([])
136         return image, label
137
138     dataset = dataset.map(map_and_set_shape, num_parallel_calls=AUTOTUNE)
139
140     # Filter out any images that were marked as invalid by load_image
141     dataset = dataset.filter(lambda x, y: tf.not_equal(y, -1))
142
143     dataset = dataset.batch(BATCH_SIZE).prefetch(AUTOTUNE)
144     return dataset
145
146 print("Creating TF Dataset pipelines...")
147 train_dataset = create_tf_dataset(train_df, shuffle=True)
148 val_dataset = create_tf_dataset(val_df)
149 test_dataset = create_tf_dataset(test_df)
150 print("Dataset pipelines ready.")
151 print("-" * 30)

```

```

149 # Define and Train the Model
150 print("Defining and compiling the Xception model...")
151 base_model = Xception(weights='imagenet', include_top=False, input_shape=
    TARGET_SIZE + (3,))
152 base_model.trainable = False
153
154 x = base_model.output
155 x = GlobalAveragePooling2D()(x)
156 x = Dense(128, activation='relu')(x)
157 x = Dropout(0.5)(x)
158 predictions = Dense(1, activation='sigmoid')(x)
159
160 model = Model(inputs=base_model.input, outputs=predictions)
161 model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'
    ])
162 model.summary()
163
164 # Define a custom callback to monitor memory usage
165 class MemoryUsageCallback(tf.keras.callbacks.Callback):
166     def on_train_begin(self, logs=None):
167         self.memory_usage = []
168     def on_epoch_end(self, epoch, logs=None):
169         process = psutil.Process(os.getpid())
170         mem_gb = process.memory_info().rss / (1024 ** 3)
171         self.memory_usage.append(mem_gb)
172         print(f"Epoch {epoch+1}: Process memory usage: {mem_gb:.2f} GB")
173
174 # Define callbacks
175 model_checkpoint_path = os.path.join(MODELS_PATH, 'xception_tfdata_best_model.h5
    ')
176 checkpoint = ModelCheckpoint(model_checkpoint_path, monitor='val_accuracy',
    save_best_only=True, mode='max', verbose=1)
177 early_stopping = EarlyStopping(monitor='val_loss', patience=3, verbose=1)
178 mem_callback = MemoryUsageCallback()
179
180 print("\nStarting model training...")
181 start_time = time.time()
182 history = model.fit(
183     train_dataset,
184     epochs=EPOCHS,
185     validation_data=val_dataset,
186     callbacks=[checkpoint, early_stopping, mem_callback]
187 )
188 training_time = time.time() - start_time
189 print(f"\nTraining completed in {training_time:.2f} seconds.")
190
191 # Evaluate the Model
192 print("Loading best model and evaluating on the test set...")
193 best_model = tf.keras.models.load_model(model_checkpoint_path)

```



```

194
195 # The inference time is captured here
196 inference_start_time = time.time()
197 test_pred_probs = best_model.predict(test_dataset)
198 inference_time = time.time() - inference_start_time
199
200 true_labels = np.concatenate([y.numpy() for x, y in test_dataset], axis=0)
201 test_preds = (test_pred_probs > 0.5).astype(int)
202
203 acc = accuracy_score(true_labels, test_preds)
204 prec = precision_score(true_labels, test_preds)
205 rec = recall_score(true_labels, test_preds)
206 f1 = f1_score(true_labels, test_preds)
207 conf_matrix = confusion_matrix(true_labels, test_preds)
208
209 results = {
210     'model': 'Xception_TFData', 'accuracy': float(acc), 'precision': float(prec)
211     ,
212     'recall': float(rec), 'f1_score': float(f1), 'confusion_matrix': conf_matrix
213     .tolist(),
214     'memory_usage_gb': mem_callback.memory_usage,
215     'inference_time_sec': float(inference_time)
216 }
217
218 results_path = os.path.join(RESULTS_PATH, 'xception_tfddata_results.json')
219 with open(results_path, 'w') as f:
220     json.dump(results, f, indent=4)
221 print(f"Evaluation results saved to: {results_path}")
222 print("-" * 30)
223
224 # Plotting the Results
225 print("Plotting training history...")
226 plt.style.use('seaborn-v0_8-whitegrid')
227 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
228 fig.suptitle('Model Performance Over Epochs', fontsize=16)
229 ax1.plot(history.history['accuracy'])
230 ax1.plot(history.history['val_accuracy'])
231 ax1.set_title('Model Accuracy')
232 ax1.set_ylabel('Accuracy')
233 ax1.set_xlabel('Epoch')
234 ax1.legend(['Train', 'Validation'], loc='upper left')
235 ax2.plot(history.history['loss'])
236 ax2.plot(history.history['val_loss'])
237 ax2.set_title('Model Loss')
238 ax2.set_ylabel('Loss')
239 ax2.set_xlabel('Epoch')
240 ax2.legend(['Train', 'Validation'], loc='upper left')
241 plt.tight_layout(rect=[0, 0.03, 1, 0.95])
242 plot_path = os.path.join(RESULTS_PATH, 'xception_tfddata_training_history.png')
243 plt.savefig(plot_path)

```

```
241 print(f"Plot saved to: {plot_path}")
```

Listing 8: Hybrid Model Detection FaceForensics++

```
1 from google.colab import drive
2 drive.flush_and_unmount()
3 drive.mount('/content/drive')
4
5 !pip install tensorflow keras scikit-learn matplotlib tqdm pillow
6
7 import os
8 import numpy as np
9 import pandas as pd
10 import tensorflow as tf
11 from tensorflow.keras.models import Model
12 from tensorflow.keras.layers import Input, TimeDistributed,
    GlobalAveragePooling2D, LSTM, Dense, Dropout
13 from tensorflow.keras.applications import Xception
14 from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping
15 from sklearn.model_selection import train_test_split
16 from tqdm import tqdm
17 import time
18 import psutil
19 import json
20 from concurrent.futures import ProcessPoolExecutor, as_completed
21
22 # Mount Google Drive
23 from google.colab import drive
24 drive.mount('/content/drive')
25
26 # Defining Paths
27 BASE_DIR = '/content/drive/MyDrive/Dissertation'
28 DATASET_FOLDER = 'ff_unified'
29 DATASET_PATH = os.path.join(BASE_DIR, DATASET_FOLDER)
30 MODELS_PATH = os.path.join(BASE_DIR, 'Models')
31 RESULTS_PATH = os.path.join(BASE_DIR, 'Results')
32
33 os.makedirs(MODELS_PATH, exist_ok=True)
34 os.makedirs(RESULTS_PATH, exist_ok=True)
35
36 # Defining Parameters
37 SEQUENCE_LENGTH = 20
38 STRIDE = 10
39 BATCH_SIZE = 4
40 TARGET_SIZE = (299, 299)
41 EPOCHS = 10
42 AUTOTUNE = tf.data.AUTOTUNE
43 NUM_WORKERS = os.cpu_count() // 2 or 1 # Use half of the available CPU cores
44
```

```

45 # Collect frames
46 def find_video_folders(base_dir):          # finds all the subdirectories containing
      image files
47
48 video_folders = []
49 for root, dirs, files in os.walk(base_dir):
50     image_files = [f for f in files if f.lower().endswith(('.png', '.jpg', '
      .jpeg'))]
51     if image_files:
52         video_folders.append((root, image_files))
53 return video_folders
54
55 def process_single_video_folder(folder_info):    # Processes a single video
      folder to create sequences.
56
57 folder, frames = folder_info
58 full_frames_paths = [os.path.join(folder, f) for f in sorted(frames)]
59 num_frames = len(full_frames_paths)
60 label = 1 if 'real' in folder else 0
61
62 sequences = []
63 labels = []
64
65 for start in range(0, num_frames - SEQUENCE_LENGTH + 1, STRIDE):
66     sequence = full_frames_paths[start:start + SEQUENCE_LENGTH]
67     sequences.append(sequence)
68     labels.append(label)
69
70 return sequences, labels
71
72 def collect_all_sequences_multiprocessing(base_dir):
73     all_sequences = []
74     all_labels = []
75
76     print("Checking if dataset path exists...")
77     if not os.path.exists(base_dir):
78         print(f"Error: Dataset path '{base_dir}' does not exist.")
79         return [], []
80     print("Dataset path found. Starting a single pass over the directory.")
81
82     video_folders = find_video_folders(base_dir)
83     print(f"Found {len(video_folders)} video folders with image files.")
84
85     # Use ProcessPoolExecutor to parallelize the sequence creation
86     with ProcessPoolExecutor(max_workers=NUM_WORKERS) as executor:
87         futures = {executor.submit(process_single_video_folder, folder): folder
      for folder in video_folders}
88
89     for future in tqdm(as_completed(futures), total=len(video_folders), desc

```

```

    ="Creating sequences (multiprocessing)":
90     try:
91         sequences, labels = future.result()
92         all_sequences.extend(sequences)
93         all_labels.extend(labels)
94     except Exception as exc:
95         print(f'A folder generated an exception: {exc}')
96
97     return all_sequences, all_labels
98
99 print("Starting data collection...")
100 all_sequences, all_labels = collect_all_sequences_multiprocessing(DATASET_PATH)
101
102 print(f"\nTotal sequences created: {len(all_sequences)}")
103 if not all_sequences:
104     print("Error: No sequences were created. Please check your DATASET_PATH and
105         folder structure.")
106     exit()
107
108 df = pd.DataFrame({'frame_sequence': all_sequences, 'label': all_labels})
109
110 # Split dataset
111 train_df, temp_df = train_test_split(df, test_size=0.3, random_state=42,
112     stratify=df['label'])
113 val_df, test_df = train_test_split(temp_df, test_size=0.5, random_state=42,
114     stratify=temp_df['label'])
115
116 print(f"Training: {len(train_df)}, Validation: {len(val_df)}, Testing: {len(
117     test_df)}")
118 print("-" * 30)
119
120 # TF Dataset
121 def load_sequence(frames, label):
122     images = []
123     for path in frames:
124         try:
125             img = tf.io.read_file(path)
126             img = tf.image.decode_image(img, channels=3)
127             img = tf.image.resize(img, TARGET_SIZE)
128             img = tf.cast(img, tf.float32) / 255.0
129             images.append(img)
130         except tf.errors.InvalidArgumentError:
131             print(f"Skipping empty or invalid image file: {path}")
132             # Instead of failing, return a special value to filter later
133             return tf.constant([], dtype=tf.float32), tf.constant(-1, dtype=tf.
134                 int32)
135
136     sequence_tensor = tf.stack(images)
137     label_tensor = tf.cast(label, tf.int32)

```

```

133
134     return sequence_tensor, label_tensor
135
136 def create_tf_dataset(df, shuffle=False):
137     dataset = tf.data.Dataset.from_tensor_slices((df['frame_sequence'].tolist(),
138         df['label'].tolist()))
139
140     if shuffle:
141         dataset = dataset.shuffle(buffer_size=len(df), reshuffle_each_iteration=
142             True)
143
144     def map_and_set_shape(x, y):
145         sequence, label = tf.py_function(load_sequence, inp=[x, y],
146             Tout=[tf.float32, tf.int32])
147
148         sequence.set_shape([None, TARGET_SIZE[0], TARGET_SIZE[1], 3])
149         label.set_shape([]) # Scalar
150         return sequence, label
151
152     dataset = dataset.map(map_and_set_shape, num_parallel_calls=AUTOTUNE)
153
154     # Filter out any sequences that were marked as invalid by load_sequence
155     dataset = dataset.filter(lambda x, y: tf.not_equal(y, -1))
156
157     # After filtering, we can set the shape to be a fixed sequence length
158     def set_final_shape(x, y):
159         x.set_shape([SEQUENCE_LENGTH, TARGET_SIZE[0], TARGET_SIZE[1], 3])
160         return x, y
161
162     dataset = dataset.map(set_final_shape)
163
164     dataset = dataset.batch(BATCH_SIZE).prefetch(AUTOTUNE)
165     return dataset
166
167 print("Creating TF Dataset pipelines...")
168 train_dataset = create_tf_dataset(train_df, shuffle=True)
169 val_dataset = create_tf_dataset(val_df)
170 test_dataset = create_tf_dataset(test_df)
171
172 print("Dataset pipelines ready.")
173 print("-" * 30)
174
175 # Build hybrid model
176 xception_base = Xception(weights='imagenet', include_top=False, input_shape=
177     TARGET_SIZE + (3,))
178 xception_base.trainable = False
179 input_layer = Input(shape=(SEQUENCE_LENGTH,) + TARGET_SIZE + (3,))
180 x = TimeDistributed(xception_base)(input_layer)
181 x = TimeDistributed(GlobalAveragePooling2D())(x)

```

```

179 x = LSTM(128, return_sequences=False)(x)
180 x = Dropout(0.5)(x)
181 output_layer = Dense(1, activation='sigmoid')(x)
182 hybrid_model = Model(inputs=input_layer, outputs=output_layer)
183 hybrid_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['
    accuracy'])
184 hybrid_model.summary()
185
186 # Defining callbacks
187 checkpoint_path = os.path.join(MODELS_PATH, 'hybrid_xception_lstm_best.h5')
188 checkpoint = ModelCheckpoint(checkpoint_path, monitor='val_accuracy',
    save_best_only=True, verbose=1)
189 early_stopping = EarlyStopping(monitor='val_loss', patience=3,
    restore_best_weights=True, verbose=1)
190
191 class MemoryCallback(tf.keras.callbacks.Callback):
192     def on_train_begin(self, logs=None):
193         self.memory_usage = []
194     def on_epoch_end(self, epoch, logs=None):
195         process = psutil.Process(os.getpid())
196         mem_gb = process.memory_info().rss / (1024 ** 3)
197         self.memory_usage.append(mem_gb)
198         print(f"Epoch {epoch+1} memory usage: {mem_gb:.2f} GB")
199
200 mem_callback = MemoryCallback()
201
202 # Train the model
203 print("\nStarting training...")
204 start_time = time.time()
205 history = hybrid_model.fit(train_dataset, epochs=EPOCHS,
206                             validation_data=val_dataset,
207                             callbacks=[checkpoint, early_stopping, mem_callback])
208 training_time = time.time() - start_time
209 print(f"\nTraining completed in {training_time:.2f} seconds.")
210
211 # Evaluate the model
212 print("\nLoading best model for evaluation...")
213 best_model = tf.keras.models.load_model(checkpoint_path)
214
215 # Measure inference time
216 start_inf = time.time()
217 test_pred_probs = best_model.predict(test_dataset)
218 inference_time = time.time() - start_inf
219
220 test_preds = (test_pred_probs > 0.5).astype(int)
221 true_labels = np.concatenate([y for x, y in test_dataset], axis=0)
222 from sklearn.metrics import accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix
223 acc = accuracy_score(true_labels, test_preds)

```

```

224 prec = precision_score(true_labels, test_preds)
225 rec = recall_score(true_labels, test_preds)
226 f1 = f1_score(true_labels, test_preds)
227 conf_matrix = confusion_matrix(true_labels, test_preds)
228 results = {
229     'model': 'Hybrid_Xception_LSTM',
230     'accuracy': float(acc),
231     'precision': float(prec),
232     'recall': float(rec),
233     'f1_score': float(f1),
234     'confusion_matrix': conf_matrix.tolist(),
235     'training_time_sec': training_time,
236     'inference_time_sec': inference_time,
237     'memory_usage_gb': mem_callback.memory_usage
238 }
239 results_path = os.path.join(RESULTS_PATH, 'hybrid_xception_lstm_results.json')
240 with open(results_path, 'w') as f:
241     json.dump(results, f, indent=4)
242 print(f"\nResults saved to: {results_path}")
243
244 print("\nGenerating training history plots...")
245
246 import matplotlib.pyplot as plt
247
248 # Get training and validation metrics from the history object
249 acc = history.history['accuracy']
250 val_acc = history.history['val_accuracy']
251 loss = history.history['loss']
252 val_loss = history.history['val_loss']
253
254 epochs_range = range(len(acc))
255
256 # Plot Training and Validation Accuracy
257 plt.figure(figsize=(12, 6))
258 plt.subplot(1, 2, 1)
259 plt.plot(epochs_range, acc, label='Training Accuracy')
260 plt.plot(epochs_range, val_acc, label='Validation Accuracy')
261 plt.legend(loc='lower right')
262 plt.title('Training and Validation Accuracy')
263 plt.xlabel('Epoch')
264 plt.ylabel('Accuracy')
265 plt.grid(True)
266
267 # Plot Training and Validation Loss
268 plt.subplot(1, 2, 2)
269 plt.plot(epochs_range, loss, label='Training Loss')
270 plt.plot(epochs_range, val_loss, label='Validation Loss')
271 plt.legend(loc='upper right')
272 plt.title('Training and Validation Loss')

```

```

273 plt.xlabel('Epoch')
274 plt.ylabel('Loss')
275 plt.grid(True)
276
277 plt.tight_layout()
278
279 # Save the plots to a file
280 plot_path = os.path.join(RESULTS_PATH, 'training_history.png')
281 plt.savefig(plot_path)
282 print(f"Training history plots saved to: {plot_path}")
283
284 plt.show()

```

Listing 9: Operation Count per forward pass (FLOPs)

```

1  from google.colab import drive
2  drive.mount('/content/drive')
3
4  !pip install tensorflow keras
5
6  #Celeb-DF CNN
7
8  import pandas as pd
9  from tensorflow.keras.models import load_model
10 from tensorflow.keras.layers import Conv2D, SeparableConv2D
11 import math
12
13 # Load saved model
14 model = load_model('/content/drive/MyDrive/Dissertation/Models/celeb_df_CNN/
    xception_best_model_multithreaded.h5')
15
16 # Extract convolution layers with unified FLOP formuls
17 layer_data = []
18 for layer in model.layers:
19     if isinstance(layer, (Conv2D, SeparableConv2D)):
20         config = layer.get_config()
21         filters = config.get("filters", None)
22         kernel_size = config.get("kernel_size", (1, 1))
23         strides = config.get("strides", (1, 1))
24
25         # Output shape
26         output_shape = [dim if dim is not None else -1 for dim in layer.output.
            shape]
27         if len(output_shape) == 4:
28             H, W, C_out = output_shape[1], output_shape[2], output_shape[3]
29         else:
30             H, W, C_out = -1, -1, -1
31
32         # Input channels

```



```

33     try:
34         C_in = int(layer.input.shape[-1])
35     except:
36         C_in = None
37
38     # Unified operation estimate ---
39     ops = None
40     if H != -1 and W != -1 and C_in is not None and C_out is not None:
41         K = kernel_size[0]
42         ops = H * W * C_in * (K**2 + C_out)
43
44     layer_data.append({
45         "Layer Name": layer.name,
46         "Layer Type": type(layer).__name__,
47         "Filters": filters,
48         "Kernel Size": kernel_size,
49         "Strides": strides,
50         "Input Channels": C_in,
51         "Output Shape": tuple(output_shape),
52         "Operation Count (FLOPs)": ops if ops is not None else 0
53     })
54
55 # Put into DataFrame
56 df_layers = pd.DataFrame(layer_data)
57
58 # Add TOTAL row
59 total_ops = df_layers["Operation Count (FLOPs)"].sum()
60 df_total = pd.DataFrame([
61     "Layer Name": "TOTAL",
62     "Layer Type": "",
63     "Filters": "",
64     "Kernel Size": "",
65     "Strides": "",
66     "Input Channels": "",
67     "Output Shape": "",
68     "Operation Count (FLOPs)": total_ops
69 ])
70
71 df_layers = pd.concat([df_layers, df_total], ignore_index=True)
72
73 # Format FLOPs in scientific notation
74 def format_ops(x):
75     if isinstance(x, (int, float)) and x != 0:
76         exponent = int(math.floor(math.log10(x)))
77         coeff = x / (10**exponent)
78         return f"{coeff:.2f} × 10{exponent}"
79     return x
80
81 df_layers["Operation Count (FLOPs)"] = df_layers["Operation Count (FLOPs)"].

```

```

    apply(format_ops)
82
83 # Add Theoretical Big-O Summary Rows
84 big_o_summary = pd.DataFrame([
85     {
86         "Layer Name": "Big-O (Training)",
87         "Operation Count (FLOPs)": "O(E × N × F)"
88     },
89     {
90         "Layer Name": "Big-O (Inference per sample)",
91         "Operation Count (FLOPs)": "O(F)"
92     }
93 ])
94
95 df_layers = pd.concat([df_layers, big_o_summary], ignore_index=True)
96
97 # save results to excel file
98 output_file = "xception_big_o_with_summary.xlsx"
99 df_layers.to_excel(output_file, index=False)
100
101 print(f"Big-O summary + operation counts saved to {output_file}")
102
103 # FF++ CNN
104
105 import pandas as pd
106 from tensorflow.keras.models import load_model
107 from tensorflow.keras.layers import Conv2D, SeparableConv2D
108 import math
109
110 # Load saved model
111 model = load_model('/content/drive/MyDrive/Dissertation/Models/FF_CNN_model/
    xception_tfdata_best_model.h5')
112
113 # Extract convolution layers with unified FLOP formula
114 layer_data = []
115 for layer in model.layers:
116     if isinstance(layer, (Conv2D, SeparableConv2D)):
117         config = layer.get_config()
118         filters = config.get("filters", None)
119         kernel_size = config.get("kernel_size", (1, 1))
120         strides = config.get("strides", (1, 1))
121
122         # Output shape
123         output_shape = [dim if dim is not None else -1 for dim in layer.output.
            shape]
124         if len(output_shape) == 4:
125             H, W, C_out = output_shape[1], output_shape[2], output_shape[3]
126         else:
127             H, W, C_out = -1, -1, -1

```

```

128
129     # Input channels
130     try:
131         C_in = int(layer.input.shape[-1])
132     except:
133         C_in = None
134
135     # Unified operation estimate
136     ops = None
137     if H != -1 and W != -1 and C_in is not None and C_out is not None:
138         K = kernel_size[0]
139         ops = H * W * C_in * (K**2 + C_out)
140
141     layer_data.append({
142         "Layer Name": layer.name,
143         "Layer Type": type(layer).__name__,
144         "Filters": filters,
145         "Kernel Size": kernel_size,
146         "Strides": strides,
147         "Input Channels": C_in,
148         "Output Shape": tuple(output_shape),
149         "Operation Count (FLOPs)": ops if ops is not None else 0
150     })
151
152     # Put into DataFrame
153     df_layers = pd.DataFrame(layer_data)
154
155     # Add TOTAL row
156     total_ops = df_layers["Operation Count (FLOPs)"].sum()
157     df_total = pd.DataFrame([
158         "Layer Name": "TOTAL",
159         "Layer Type": "",
160         "Filters": "",
161         "Kernel Size": "",
162         "Strides": "",
163         "Input Channels": "",
164         "Output Shape": "",
165         "Operation Count (FLOPs)": total_ops
166     ])
167
168     df_layers = pd.concat([df_layers, df_total], ignore_index=True)
169
170     # Format FLOPs in scientific notation
171     def format_ops(x):
172         if isinstance(x, (int, float)) and x != 0:
173             exponent = int(math.floor(math.log10(x)))
174             coeff = x / (10**exponent)
175             return f"{coeff:.2f} × 10{exponent}"
176         return x

```

```

177
178 df_layers["Operation Count (FLOPs)"] = df_layers["Operation Count (FLOPs)"].
    apply(format_ops)
179
180 # Add Theoretical Big-O Summary Rows
181 big_o_summary = pd.DataFrame([
182     {
183         "Layer Name": "Big-O (Training)",
184         "Operation Count (FLOPs)": "O(E × N × F)"
185     },
186     {
187         "Layer Name": "Big-O (Inference per sample)",
188         "Operation Count (FLOPs)": "O(F)"
189     }
190 ])
191
192 df_layers = pd.concat([df_layers, big_o_summary], ignore_index=True)
193
194 # save results to excel file
195 output_file = "xception_ops_count_ff.xlsx"
196 df_layers.to_excel(output_file, index=False)
197
198 print(f"Big-O summary + operation counts saved to {output_file}")
199
200 # Hybrid model celeb-df
201
202 import pandas as pd
203 from tensorflow.keras.models import load_model
204 from tensorflow.keras.layers import Conv2D, SeparableConv2D, Dense, LSTM
205 import math
206
207 # Load saved model
208 model = load_model('/content/drive/MyDrive/Dissertation/Models/
    celeb_df_hybrid_model/hybrid_xception_lstm_model.h5')
209
210 # Extract relevant layers
211 layer_data = []
212 for layer in model.layers:
213     config = layer.get_config()
214
215     # Convolutional Layers
216     if isinstance(layer, (Conv2D, SeparableConv2D)):
217         filters = config.get("filters", None)
218         kernel_size = config.get("kernel_size", (1, 1))
219         strides = config.get("strides", (1, 1))
220
221         output_shape = [dim if dim is not None else -1 for dim in layer.output.
            shape]
222         if len(output_shape) == 4:

```

```

223         H, W, C_out = output_shape[1], output_shape[2], output_shape[3]
224     else:
225         H, W, C_out = -1, -1, -1
226
227     try:
228         C_in = int(layer.input.shape[-1])
229     except:
230         C_in = None
231
232     ops = None
233     if H != -1 and W != -1 and C_in is not None and C_out is not None:
234         K = kernel_size[0]
235         ops = H * W * C_in * (K**2 + C_out)
236
237     layer_data.append({
238         "Layer Name": layer.name,
239         "Layer Type": type(layer).__name__,
240         "Filters": filters,
241         "Kernel Size": kernel_size,
242         "Strides": strides,
243         "Input Channels": C_in,
244         "Output Shape": tuple(output_shape),
245         "Operation Count (FLOPs)": ops if ops is not None else 0
246     })
247
248     # Dense Layers
249     elif isinstance(layer, Dense):
250         units = config.get("units", None)
251         input_dim = int(layer.input.shape[-1])
252
253         ops = input_dim * units
254
255         layer_data.append({
256             "Layer Name": layer.name,
257             "Layer Type": "Dense",
258             "Filters": "",
259             "Kernel Size": "",
260             "Strides": "",
261             "Input Channels": input_dim,
262             "Output Shape": tuple(layer.output.shape),
263             "Operation Count (FLOPs)": ops
264         })
265
266     # LSTM Layers
267     elif isinstance(layer, LSTM):
268         units = config.get("units", None)
269         timesteps = layer.input.shape[1]
270         input_dim = layer.input.shape[2]

```

```

272     # Standard LSTM FLOPs estimate:
273     ops = 4 * units * (input_dim + units) * timesteps
274
275     layer_data.append({
276         "Layer Name": layer.name,
277         "Layer Type": "LSTM",
278         "Filters": "",
279         "Kernel Size": "",
280         "Strides": "",
281         "Input Channels": input_dim,
282         "Output Shape": tuple(layer.output.shape),
283         "Operation Count (FLOPs)": ops
284     })
285
286 # Create DataFrame
287 df_layers = pd.DataFrame(layer_data)
288
289 # Add TOTAL row
290 total_ops = df_layers["Operation Count (FLOPs)"].sum()
291 df_total = pd.DataFrame([
292     "Layer Name": "TOTAL",
293     "Layer Type": "",
294     "Filters": "",
295     "Kernel Size": "",
296     "Strides": "",
297     "Input Channels": "",
298     "Output Shape": "",
299     "Operation Count (FLOPs)": total_ops
300 ])
301 df_layers = pd.concat([df_layers, df_total], ignore_index=True)
302
303 # Format FLOPs
304 def format_ops(x):
305     if isinstance(x, (int, float)) and x != 0:
306         exponent = int(math.floor(math.log10(x)))
307         coeff = x / (10**exponent)
308         return f"{coeff:.2f} × 10{exponent}"
309     return x
310
311 df_layers["Operation Count (FLOPs)"] = df_layers["Operation Count (FLOPs)"].
312     apply(format_ops)
313
314 # Add Theoretical Big-O Summary Rows
315 big_o_summary = pd.DataFrame([
316     {"Layer Name": "Big-O (Training)", "Operation Count (FLOPs)": "O(E × N × F)"
317     },
318     {"Layer Name": "Big-O (Inference per sample)", "Operation Count (FLOPs)": "O
319     (F)"}
320 ])

```

```

318 df_layers = pd.concat([df_layers, big_o_summary], ignore_index=True)
319
320 # save to excel file
321 output_file = "hybrid_xception_lstm_ops.xlsx"
322 df_layers.to_excel(output_file, index=False)
323
324 print(f"Big-0 summary + operation counts saved to {output_file}")
325
326 # Hybrid model FF++
327
328 import pandas as pd
329 from tensorflow.keras.models import load_model
330 from tensorflow.keras.layers import Conv2D, SeparableConv2D, Dense, LSTM
331 import math
332
333 # Load saved model
334 model = load_model('/content/drive/MyDrive/Dissertation/Models/FF_hybrid_model/
    hybrid_xception_lstm_best_ff.h5')
335
336 # Extract relevant layers
337 layer_data = []
338 for layer in model.layers:
339     config = layer.get_config()
340
341     # Convolutional Layers
342     if isinstance(layer, (Conv2D, SeparableConv2D)):
343         filters = config.get("filters", None)
344         kernel_size = config.get("kernel_size", (1, 1))
345         strides = config.get("strides", (1, 1))
346
347         output_shape = [dim if dim is not None else -1 for dim in layer.output.
            shape]
348         if len(output_shape) == 4:
349             H, W, C_out = output_shape[1], output_shape[2], output_shape[3]
350         else:
351             H, W, C_out = -1, -1, -1
352
353         try:
354             C_in = int(layer.input.shape[-1])
355         except:
356             C_in = None
357
358         ops = None
359         if H != -1 and W != -1 and C_in is not None and C_out is not None:
360             K = kernel_size[0]
361             ops = H * W * C_in * (K**2 + C_out)
362
363         layer_data.append({
364             "Layer Name": layer.name,

```

```

365         "Layer Type": type(layer).__name__,
366         "Filters": filters,
367         "Kernel Size": kernel_size,
368         "Strides": strides,
369         "Input Channels": C_in,
370         "Output Shape": tuple(output_shape),
371         "Operation Count (FLOPs)": ops if ops is not None else 0
372     })
373
374     # Dense Layers
375     elif isinstance(layer, Dense):
376         units = config.get("units", None)
377         input_dim = int(layer.input.shape[-1])
378
379         ops = input_dim * units
380
381         layer_data.append({
382             "Layer Name": layer.name,
383             "Layer Type": "Dense",
384             "Filters": "",
385             "Kernel Size": "",
386             "Strides": "",
387             "Input Channels": input_dim,
388             "Output Shape": tuple(layer.output.shape),
389             "Operation Count (FLOPs)": ops
390         })
391
392     # LSTM Layers
393     elif isinstance(layer, LSTM):
394         units = config.get("units", None)
395         timesteps = layer.input.shape[1]
396         input_dim = layer.input.shape[2]
397
398         # Standard LSTM FLOPs estimate:
399         ops = 4 * units * (input_dim + units) * timesteps
400
401         layer_data.append({
402             "Layer Name": layer.name,
403             "Layer Type": "LSTM",
404             "Filters": "",
405             "Kernel Size": "",
406             "Strides": "",
407             "Input Channels": input_dim,
408             "Output Shape": tuple(layer.output.shape),
409             "Operation Count (FLOPs)": ops
410         })
411
412     # Create DataFrame
413     df_layers = pd.DataFrame(layer_data)

```



```

414
415 # Add TOTAL row
416 total_ops = df_layers["Operation Count (FLOPs)"].sum()
417 df_total = pd.DataFrame([
418     "Layer Name": "TOTAL",
419     "Layer Type": "",
420     "Filters": "",
421     "Kernel Size": "",
422     "Strides": "",
423     "Input Channels": "",
424     "Output Shape": "",
425     "Operation Count (FLOPs)": total_ops
426 ])
427 df_layers = pd.concat([df_layers, df_total], ignore_index=True)
428
429 # Format FLOPs
430 def format_ops(x):
431     if isinstance(x, (int, float)) and x != 0:
432         exponent = int(math.floor(math.log10(x)))
433         coeff = x / (10**exponent)
434         return f"{coeff:.2f} × 10{exponent}"
435     return x
436
437 df_layers["Operation Count (FLOPs)"] = df_layers["Operation Count (FLOPs)"].
438     apply(format_ops)
439
440 # Add Theoretical Big-O Summary Rows
441 big_o_summary = pd.DataFrame([
442     {"Layer Name": "Big-O (Training)", "Operation Count (FLOPs)": "O(E × N × F)"
443     },
444     {"Layer Name": "Big-O (Inference per sample)", "Operation Count (FLOPs)": "O
445     (F)"}
446 ])
447 df_layers = pd.concat([df_layers, big_o_summary], ignore_index=True)
448
449 # save to Excel
450 output_file = "hybrid_xception_lstm_ff_ops.xlsx"
451 df_layers.to_excel(output_file, index=False)
452
453 print(f"Big-O summary + operation counts saved to {output_file}")

```